

PRACA DYPLOMOWA MAGISTERSKA

Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach

*A performance and efficiency analysis of selected
parallel sorting algorithms on multicore processors*

Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

Niniejszym chciałbym serdecznie podziękować

...

*za bezcenne wsparcie udzielone mi
w trakcie trwania studiów.*

Spis treści

1	Wstęp	3
1.1	Cel pracy	4
1.2	Zakres pracy	4
2	Wprowadzenie teoretyczne	7
2.1	Wprowadzenie do sortowania równoległego	7
2.2	Charakterystyka procesorów wielordzeniowych	8
2.3	Wydajność i efektywność	10
2.4	Skalowalność algorytmów równoległych	11
3	Opis wybranych algorytmów sortowania równoległego	13
3.1	Parallel Quicksort	13
3.2	Parallel Merge Sort	14
3.3	Parallel Bucket Sort	16
3.4	Parallel Samplesort	16
4	Środowisko badawcze i implementacja	19
4.1	Opis środowiska	19
4.2	Wybór języka i bibliotek	20
4.3	Implementacja wybranych algorytmów	21
4.4	Charakterystyka danych testowych	27
5	Metodyka badań	31
5.1	Scenariusze testowe	31
5.2	Wariant sekwencyjny jako punkt odniesienia	32
5.3	Parametry pomiarów	33
5.4	Narzędzia do pomiaru czasu, CPU i pamięci	34
5.5	Przebieg eksperymentów	35
6	Analiza wyników badań	37
6.1	Porównanie czasu sortowania	37
6.2	Wpływ liczby jednostek wykonawczych na wydajność	39
6.3	Analiza zużycia pamięci RAM i obciążenia CPU	41

6.4	Skalowalność algorytmów	46
6.5	Wpływ typu i charakterystyki danych wejściowych	52
7	Wnioski	59
7.1	Najważniejsze wyniki przeprowadzonych badań	59
7.2	Interpretacja uzyskanych wyników	59
7.3	Ograniczenia przeprowadzonych badań	59
7.4	Kierunki dalszych badań	59
	Podsumowanie	61
	Bibliografia	63
	Spis rysunków	66
	Spis tabel	67
	Listing	68
	Streszczenie	69
	Summary	71
	Słowa kluczowe	73
	Keywords	75

Rozdział 1

Wstęp

W ostatnim czasie można zaobserwować intensywny rozwój architektury procesorów wielordzeniowych. Ograniczenia związane z dalszym zwiększaniem wydajności pojedynczych rdzeni przyczyniły się do rozwoju procesorów wykorzystujących wiele rdzeni fizycznych oraz logicznych. Taka zmiana podejścia projektowania procesorów stawia przed programistami nowe wyzwania. W konsekwencji coraz większego znaczenia nabiera tworzenie oprogramowania umożliwiającego równoległą realizację obliczeń oraz efektywne wykorzystanie dostępnych zasobów sprzętowych [11].

Szczególnie ważnym obszarem, w którym równoległość może przynieść wymierne korzyści, jest sortowanie danych. Sortowanie stanowi jeden z fundamentalnych problemów informatyki i znajduje szerokie zastosowanie m.in. w bazach danych, systemach plików, przetwarzaniu dużych zbiorów danych oraz uczeniu maszynowym. Wraz ze wzrostem objętości przetwarzanych informacji klasyczne algorytmy sortowania sekwencyjnego mogą coraz częściej stać się wąskim gardłem wydajnościowym.

Implementacja efektywnych algorytmów sortowania równoległego wiąże się z szeregiem wyzwań, takich jak odpowiedni podział danych, synchronizacja jednostek wykonawczych, zarządzanie dostępem do pamięci oraz ograniczanie narzutu wynikającego z realizacji obliczeń równoległych. W praktyce wiele istniejących rozwiązań albo pozostaje na poziomie teoretycznym, albo nie uwzględnia realnych ograniczeń sprzętowych i systemowych, takich jak koszt przełączania kontekstu, konflikty w dostępie do pamięci czy nierównomierne obciążenie rdzeni. Z tego względu zwiększenie liczby jednostek wykonawczych nie musi prowadzić do proporcjonalnego skrócenia czasu wykonania algorytmu [9].

W związku z tym praktyczne zbadanie wydajności oraz efektywności wybranych algorytmów sortowania równoległego stanowi istotne i aktualne zagadnienie. W niniejszej pracy skoncentrowano się na implementacji i porównaniu czterech popularnych algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort. Algorytmy zostały zrealizowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie poddane testom pod kątem

czasu wykonania, wykorzystania pamięci RAM, obciążenia procesora, skalowalności oraz wpływu charakterystyki danych wejściowych.

1.1 Cel pracy

Głównym celem niniejszej pracy magisterskiej jest przeprowadzenie szczegółowej analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na współczesnych wielordzeniowych procesorach. Analizie podano cztery algorytmy: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort.

Realizacja głównego celu pracy opiera się na implementacji wybranych algorytmów w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie przeprowadzeniu badań eksperymentalnych w środowisku wielordzeniowym. Każdy z algorytmów zaimplementowano zarówno w wariancie sekwencyjnym, stanowiącym punkt odniesienia, jak i w wariancie równoległym, uruchamianym przy różnej liczbie wykorzystywanych rdzeni procesora.

1.2 Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu sortowania równoległego, architektury procesorów wielordzeniowych, wydajności i efektywności obliczeń równoległych oraz skalowalności algorytmów
- ▶ przedstawienie oraz porównanie wybranych algorytmów sortowania równoległego: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort
- ▶ opracowanie założeń projektowych implementacji algorytmów w środowisku wielordzeniowym z wykorzystaniem biblioteki `multiprocessing` oraz pamięci współdzielonej
- ▶ implementację wariantów sekwencyjnych i równoległych wybranych algorytmów sortowania w języku Python
- ▶ przygotowanie danych testowych o różnych rozmiarach i charakterystykach, obejmujących dane losowe, dane częściowo posortowane oraz dane zawierające duplikaty, dla liczb całkowitych i zmiennoprzecinkowych
- ▶ przeprowadzenie eksperymentów dla różnych rozmiarów danych oraz różnej liczby wykorzystywanych rdzeni procesora

- ▶ pomiar czasu wykonania, obciążenia procesora oraz wykorzystania pamięci operacyjnej podczas działania badanych algorytmów
- ▶ wyznaczenie i analizę metryk *speedup* (przyspieszenie) oraz *efficiency* (efektywność)
- ▶ ocenę skalowalności badanych algorytmów oraz analizę wpływu charakterystyki danych wejściowych na uzyskiwane wyniki
- ▶ analizę czynników ograniczających wydajność badanych algorytmów równoległych
- ▶ prezentację oraz analizę uzyskanych wyników za pomocą tabel i wykresów

————— Do uzupełnienia na końcu pracy —————

W rozdziale 2 przedstawiono podstawy teoretyczne sortowania równoległego, charakterystykę procesorów wielordzeniowych oraz metryki wykorzystywane do oceny wydajności i skalowalności. W rozdziale 3 opisano wybrane algorytmy sortowania równoległego. W rozdziale 4 scharakteryzowano środowisko badawcze, zastosowane technologie oraz sposób implementacji algorytmów. W rozdziale 5 przedstawiono metodykę badań, scenariusze testowe oraz narzędzia pomiarowe. W rozdziale 6 zaprezentowano i przeanalizowano wyniki przeprowadzonych eksperymentów. Pracę zamyka podsumowanie zawierające najważniejsze wnioski oraz możliwe kierunki dalszych badań.

Rozdział 2

Wprowadzenie teoretyczne

2.1 Wprowadzenie do sortowania równoległego

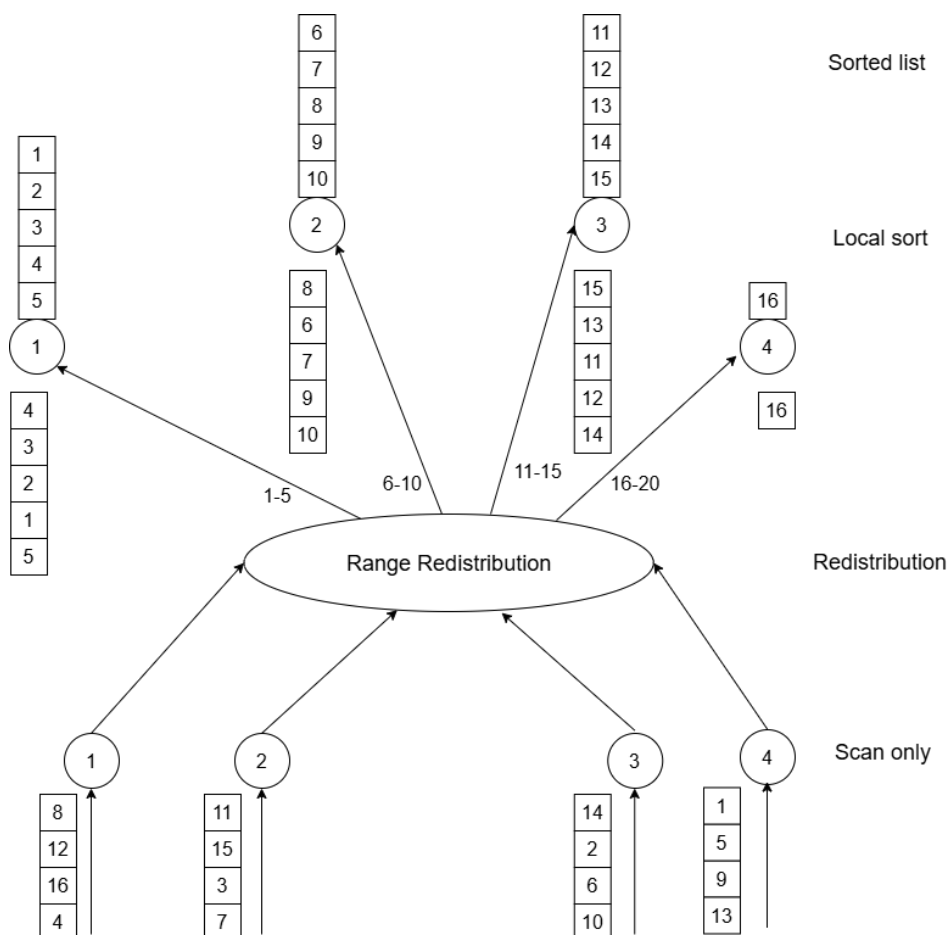
Sortowanie jest jedną z najczęściej wykonywanych operacji w informatyce. Uporządkowane dane są łatwiejsze do dalszego przetwarzania, dlatego wiele algorytmów wymaga wcześniejszego posortowania danych wejściowych. W kontekście obliczeń równoległych sortowanie ma dodatkowe znaczenie ze względu na jego ścisły związek z przemieszczaniem i dystrybucją danych pomiędzy jednostkami wykonawczymi, co stanowi istotny element wielu algorytmów równoległych [9].

Celem sortowania jest przekształcenie nieuporządkowanego ciągu elementów $S = \langle a_1, a_2, \dots, a_n \rangle$ w ciąg uporządkowany monotonicznie rosnąco lub malejąco. Algorytmy sortowania można podzielić na porównujące (ang. *comparison-based*) oraz nieporównujące (ang. *noncomparison-based*). Algorytmy porównujące ustalają kolejność elementów na podstawie wykonywanych między nimi porównań. Dolne ograniczenie złożoności sekwencyjnej dla tej klasy algorytmów wynosi $\Omega(n \log n)$. Algorytmy nieporównujące wykorzystują natomiast dodatkowe właściwości danych, takie jak ich reprezentacja lub rozkład wartości, dzięki czemu w określonych warunkach mogą osiągać złożoność liniową [9, 5].

Równoległe sortowanie polega na rozłożeniu danych oraz operacji potrzebnych do ich uporządkowania pomiędzy wiele jednostek wykonawczych. Typowym podejściem jest podział zbioru wejściowego na fragmenty, ich równoległe przetwarzanie, a następnie odpowiednia reorganizacja lub połączenie uzyskanych wyników. Sposób realizacji poszczególnych etapów zależy od zastosowanego algorytmu oraz modelu obliczeń równoległych [9].

Do najważniejszych zagadnień związanych z projektowaniem równoległych algorytmów sortowania należą sposób podziału danych, równomierne rozłożenie pracy, komunikacja pomiędzy jednostkami wykonawczymi oraz ich synchronizacja. Koszt tych operacji stanowi dodatkowy narzut, który nie występuje w takiej samej postaci w algorytmach sekwencyjnych i może ograniczać korzyści wynikające ze zrównoleglenia. W szczególności nadmierna komunikacja lub nierównomierny po-

dział pracy mogą prowadzić do sytuacji, w której część jednostek wykonawczych pozostaje bezczynna, podczas gdy pozostałe nadal wykonują obliczenia [9, 10].



Rysunek 2.1: Ogólny schemat podziału pracy w sortowaniu równoległym.

Źródło: opracowanie własne.

2.2 Charakterystyka procesorów wielordzeniowych

Rozwój współczesnych architektur procesorów jest związany między innymi z ograniczeniami dalszego zwiększania wydajności pojedynczego strumienia instrukcji. Możliwości wykorzystania równoległości na poziomie instrukcji (ang. *instruction-level parallelism*, ILP) są ograniczone, ponieważ pojedynczy wątek nie zawsze jest w stanie w pełni wykorzystać dostępne zasoby wykonawcze procesora. Jednym ze sposobów zwiększenia stopnia wykorzystania tych zasobów jest wykorzystanie równoległości na poziomie wątków (ang. *thread-level parallelism*, TLP) [11].

Procesory wielordzeniowe zawierają wiele jednostek obliczeniowych zdolnych do równoczesnego wykonywania różnych zadań. Poszczególne rdzenie mogą wykonywać niezależne strumienie instrukcji, co umożliwia jednoczesne realizowanie wielu zadań. Współczesne procesory mogą dodatkowo wykorzystywać mechanizmy

wielowątkowości sprzętowej, takie jak simultaneous multithreading (SMT). Przykładem takiego rozwiązania jest technologia Intel Hyper-Threading, w której jeden rdzeń fizyczny udostępnia dwa konteksty wykonawcze, widoczne przez system operacyjny jako dwa procesory logiczne [12, 11].

Procesory logiczne nie są jednak odpowiednikami dodatkowych niezależnych rdzeni fizycznych, ponieważ współdzielą część zasobów wykonawczych jednego rdzenia. Technologia SMT pozwala lepiej wykorzystać zasoby procesora w sytuacji, gdy pojedynczy wątek nie jest w stanie w pełni wykorzystać dostępnych jednostek wykonawczych. Uzyskiwany wzrost wydajności zależy jednak od charakterystyki obciążenia i nie musi być proporcjonalny do liczby obsługiwanych wątków sprzętowych [11].

W systemach z pamięcią współdzieloną wiele jednostek wykonawczych korzysta ze wspólnej fizycznej pamięci operacyjnej. Dane współdzielone mogą być wykorzystywane do komunikacji pomiędzy jednostkami poprzez operacje odczytu i zapisu w tej samej przestrzeni pamięci. Wykorzystanie pamięci współdzielonej wiąże się jednak z dodatkowymi wymaganiami związanymi z organizacją hierarchii pamięci oraz utrzymaniem spójnego widoku danych przechowywanych w pamięciach podręcznych poszczególnych rdzeni [11, 9].

Istotnym problemem w architekturach z pamięcią współdzieloną jest spójność pamięci podręcznych (ang. *cache coherence*). Te same dane mogą znajdować się jednocześnie w pamięciach podręcznych kilku rdzeni, dlatego zmiana wartości przez jedną jednostkę wykonawczą musi zostać odpowiednio uwzględniona przez pozostałe. Mechanizmy utrzymywania spójności mogą generować dodatkowy ruch pomiędzy pamięciami podręcznymi oraz wpływać na wydajność programów równoległych [11].

Wydajność procesora wielordzeniowego zależy nie tylko od liczby dostępnych rdzeni, lecz również od organizacji systemu pamięci. Do ważnych parametrów należą opóźnienie dostępu do pamięci (ang. *latency*) oraz jej przepustowość (ang. *bandwidth*). W przypadku aplikacji intensywnie korzystających z pamięci ograniczeniem wydajności może być szybkość dostarczania danych do jednostek wykonawczych, a nie sama moc obliczeniowa procesora [9].

Zwiększenie liczby rdzeni zwiększa dostępny poziom równoległości sprzętowej, jednak rzeczywisty wzrost wydajności zależy również od organizacji systemu pamięci, kosztów synchronizacji oraz sposobu wykorzystania dostępnych zasobów. Większa liczba rdzeni nie musi więc prowadzić do proporcjonalnego skrócenia czasu wykonania [9, 11].

2.3 Wydajność i efektywność

Podstawowymi metrykami wykorzystywanymi do oceny algorytmów równoległych są czas wykonania, przyspieszenie (ang. *speedup*) oraz efektywność (ang. *efficiency*). Umożliwiają one określenie, w jakim stopniu zastosowanie wielu jednostek wykonawczych wpływa na czas rozwiązania problemu oraz jak skutecznie wykorzystywane są dostępne zasoby obliczeniowe.

Niech $T^*(n)$ oznacza czas wykonania najlepszego znanego algorytmu sekwencyjnego dla problemu o rozmiarze n , natomiast $T_p(n)$ oznacza czas wykonania algorytmu równoległego przy wykorzystaniu p jednostek wykonawczych. Przyspieszenie definiuje się jako:

$$S_p(n) = \frac{T^*(n)}{T_p(n)}.$$

W przypadku idealnym przyspieszenie powinno być zbliżone do liczby wykorzystanych jednostek wykonawczych, czyli $S_p(n) \approx p$. W praktyce wartość ta może być mniejsza między innymi ze względu na ograniczony poziom współbieżności, synchronizację, komunikację oraz narzuty związane z realizacją obliczeń równoległych [10, 9].

Efektywność określa stopień wykorzystania dostępnych jednostek wykonawczych i jest definiowana jako stosunek uzyskanego przyspieszenia do ich liczby:

$$E_p(n) = \frac{S_p(n)}{p}.$$

Przy zastosowaniu czasu wariantu sekwencyjnego jako punktu odniesienia zależność tę można zapisać w postaci:

$$E_p(n) = \frac{T_1(n)}{p \cdot T_p(n)}.$$

Wartość efektywności bliska 1 oznacza wysokie wykorzystanie dostępnych zasobów obliczeniowych. W rzeczywistych systemach efektywność jest zazwyczaj niższa od wartości idealnej ze względu na czas poświęcany między innymi na komunikację, synchronizację oraz oczekiwanie jednostek wykonawczych [10, 9].

Istotnym ograniczeniem możliwego przyspieszenia jest udział części programu, która nie może zostać wykonana równolegle. Zależność tę opisuje prawo Amdahla. W jego podstawowym modelu czas wykonania programu dzielony jest na część sekwencyjną oraz część możliwą do wykonania równolegle. Czas części sekwencyjnej pozostaje niezależny od liczby wykorzystywanych jednostek wykonawczych, natomiast czas idealnie skalowalnej części równoległej zmniejsza się proporcjonalnie do ich liczby [13].

Jeżeli przez f oznaczony zostanie udział części sekwencyjnej programu, a przez p liczbę jednostek wykonawczych, przyspieszenie wynikające z prawa Amdahla można zapisać jako:

$$S_p = \frac{1}{f + \frac{1-f}{p}}.$$

Zależność ta wskazuje, że nawet przy zwiększaniu liczby jednostek wykonawczych część sekwencyjna programu ogranicza maksymalne możliwe przyspieszenie. Klasyczny model prawa Amdahla zakłada przy tym idealne skalowanie części równoległej i pomija dodatkowe koszty występujące w rzeczywistych systemach. Do takiego narzutu zalicza się komunikację, synchronizację oraz tworzenie procesów lub wątków [13].

2.4 Skalowalność algorytmów równoległych

Skalowalność systemu równoległego określa, jak efektywnie wykorzystuje on rosnącą liczbę jednostek wykonawczych i jaki wzrost przyspieszenia uzyskuje wraz ze zwiększaniem ich liczby. Dobrze skalowalny system powinien umożliwiać zwiększanie liczby jednostek wykonawczych bez gwałtownego spadku efektywności [9].

Przy stałym rozmiarze danych wejściowych zwiększanie liczby jednostek wykonawczych zazwyczaj powoduje stopniowe obniżanie efektywności obliczeń. Powodem tego jest wzrost całkowitego narzutu obliczeń równoległych, na który składają się takie czynniki jak komunikacja pomiędzy jednostkami, czas ich bezczynności oraz dodatkowe obliczenia wykonywane w wariancie równoległym. Wzrost liczby jednostek wykonawczych może przyczyniać się do zwiększenia tych kosztów, co skutkuje ograniczeniem uzyskiwanego przyspieszenia [9].

Skalowalność algorytmu zależy również od rozmiaru przetwarzanych danych. Dla wielu systemów równoległych zwiększenie rozmiaru danych wejściowych przy stałej liczbie jednostek wykonawczych powoduje wzrost efektywności, ponieważ większy udział w całkowitym czasie wykonania mają właściwe obliczenia. System równoległy uznaje się za skalowalny, jeżeli przy jednoczesnym zwiększaniu liczby jednostek wykonawczych oraz rozmiaru danych możliwe jest utrzymanie efektywności na zbliżonym poziomie [9].

Jedną z miar wykorzystywanych do oceny skalowalności jest funkcja *isoefficiency*, przedstawiająca zależność między wzrostem liczby jednostek wykonawczych p a rozmiarem danych W , niezbędnym do zachowania stałego poziomu efektywności. Im wolniej musi rosnąć rozmiar zadania obliczeniowego wraz ze wzrostem p , tym lepszą skalowalnością charakteryzuje się dany system równoległy. Niższe tempo wzrostu funkcji *isoefficiency* oznacza więc możliwość efektywnego wykorzystania większej liczby jednostek wykonawczych przy stosunkowo niewielkiej zmianie rozmiaru zadania obliczeniowego. [9].

Dolną granicą funkcji *isoefficiency* jest wzrost liniowy względem liczby jednostek wykonawczych. Idealnie skalowalny system charakteryzuje się zatem funkcją

isoefficiency rzędu $\Theta(p)$, co oznacza, że dla zachowania stałej efektywności wystarczający jest liniowy wzrost rozmiaru danych wejściowych wraz ze wzrostem liczby jednostek wykonawczych [9].

Analiza skalowalności umożliwia ocenę, w jaki sposób system równoległy zachowuje się przy zwiększaniu liczby jednostek wykonawczych oraz rozmiaru przetwarzanych danych.

Rozdział 3

Opis wybranych algorytmów sortowania równoległego

W niniejszym rozdziale przedstawiono wybrane algorytmy sortowania równoległego wykorzystane w dalszej części pracy. Dla każdego z nich omówiono zasadę działania, sposób podziału i łączenia danych, złożoność obliczeniową oraz najważniejsze ograniczenia związane z realizacją równoległą.

3.1 Parallel Quicksort

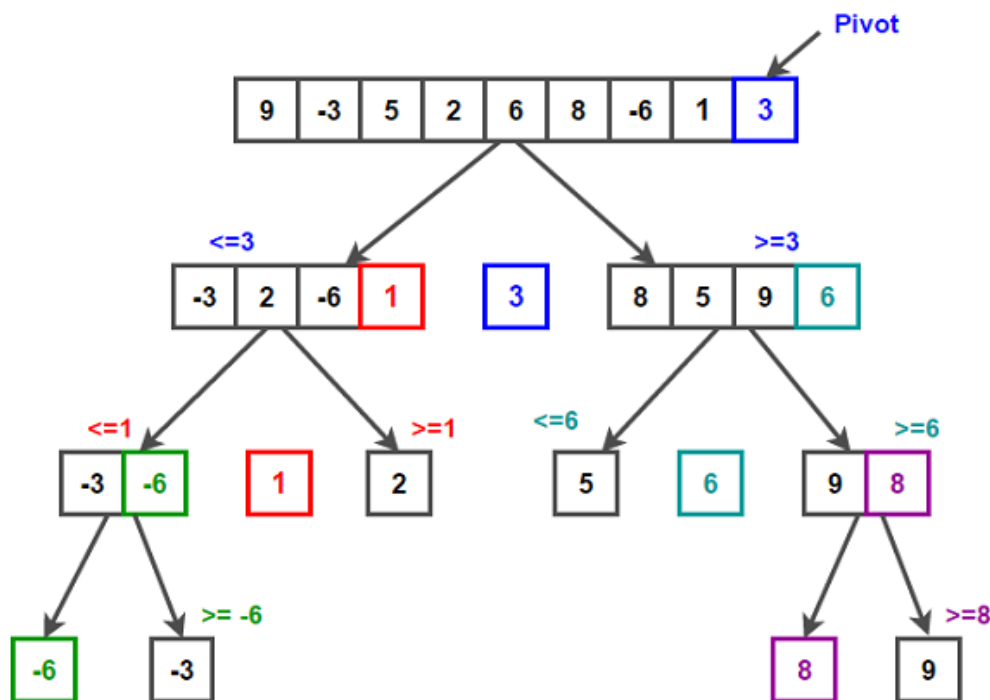
Quicksort należy do rodziny algorytmów opartych na strategii “dziel i zwyciężaj” (ang. *divide-and-conquer*) [1]. W kroku podziału wybierany jest element rozdzielający (ang. *pivot*), który dzieli wejściowy ciąg $A[p \dots r]$ na dwa podciągi. Po zakończeniu partycjonowania pivot trafia na pozycję $A[q]$, przy czym wszystkie elementy w podciągu $A[p \dots q - 1]$ są mniejsze lub równe $A[q]$, a wszystkie elementy w podciągu $A[q + 1 \dots r]$ są od niego większe. Następnie oba podciągi są rekurencyjnie sortowane aż do uzyskania podciągów o długości 1 [2].

Średnia złożoność obliczeniowa Quicksortu wynosi $O(n \log n)$, natomiast w przypadku pesymistycznym związanym z silnie niezrównoważonym podziałem może wzrosnąć do $O(n^2)$ [1].

W wersji równoległej algorytm wykorzystuje fakt, że po wykonaniu partycjonowania powstałe podciągi mogą być sortowane niezależnie od siebie. Dzięki temu możliwe jest równoległe przetwarzanie obu części tablicy przez osobne wątki. W praktycznych realizacjach poziom równoległości może być ograniczany poprzez ustalenie maksymalnej głębokości równoległej rekurencji, co pozwala ograniczyć narzut związany z tworzeniem i synchronizacją wątków. Po osiągnięciu założonej głębokości dalsze sortowanie odbywa się sekwencyjnie.

Jednym z głównych wyzwań wersji równoległej Quicksortu pozostaje nierównomierny podział danych, który występuje w sytuacji, gdy pivot jest źle dobrany.

Problem ten, w połączeniu z narzutem związanym z tworzeniem i synchronizacją wątków, może istotnie ograniczać skalowalność algorytmu [3]. Algorytm sprawdza się najlepiej przy danych zrównoważonych i dobrze dobranym pivocie, traci natomiast na efektywności m.in. gdy podziały stają się nierównomiernie.



Rysunek 3.1: Schemat działania Quicksort.

Źródło: https://medium.com/@lucky_rydar/parallel-quick-sort-on-c-a3526ba8b532.

`//medium.com/@lucky_rydar/parallel-quick-sort-on-c-a3526ba8b532`.

3.2 Parallel Merge Sort

Merge Sort należy do algorytmów opartych na strategii “dziel i zwyciężaj” (ang. *divide-and-conquer*) [5]. Algorytm dzieli wejściowy ciąg na dwa podciągi o zbliżonej długości, następnie rekurencyjnie sortuje obie części, a na końcu scala je w jeden posortowany ciąg.

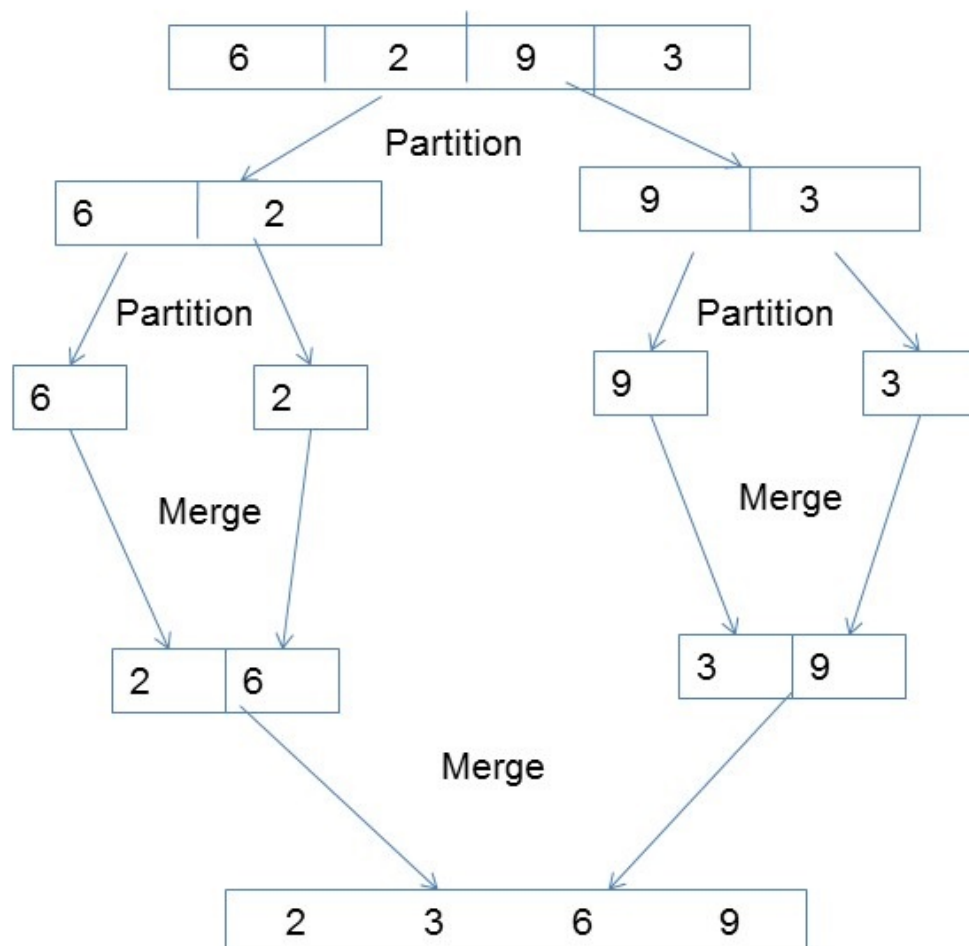
Operacja scalania stanowi kluczowy etap algorytmu i dla dwóch posortowanych podciągów o łącznej liczbie n elementów może zostać wykonana w czasie $O(n)$. Jeżeli zastosowana procedura scalania jest stabilna, cały algorytm Merge Sort również zachowuje stabilność [2].

Złożoność czasowa Merge Sort wynosi $O(n \log n)$. W przeciwieństwie do Quicksortu jej rząd nie zależy od początkowego uporządkowania danych, ponieważ algorytm w kolejnych krokach dzieli problem na dwa podproblemy o zbliżonych rozmiarach i wykonuje liniową operację scalania [5].

W wersji równoległej wykorzystuje się niezależność dwóch podproblemów powstałych po podziale danych. Rekurencyjne sortowanie obu części może dzięki

temu odbywać się równolegle. Przed rozpoczęciem scalania konieczne jest jednak zakończenie sortowania obu podciągów, ponieważ etap scalania wykorzystuje wyniki obu wcześniejszych operacji [5]. Samo scalanie może być realizowane sekwencyjnie lub równolegle.

Jednym z ograniczeń Parallel Merge Sort jest etap scalania. Wariant wykorzystujący równoległe sortowanie podciągów, lecz sekwencyjne scalanie, oferuje ograniczony poziom równoległości, ponieważ sekwencyjna operacja scalania staje się wąskim gardłem algorytmu [5]. Zastosowanie równoległego scalania pozwala zwiększyć dostępny poziom równoległości, jednak wiąże się z bardziej rozbudowanym sposobem łączenia posortowanych części. W praktycznych realizacjach dla odpowiednio małych podproblemów można również przejść do wykonania sekwencyjnego w celu ograniczenia narzutu związanego z równoległością [5].



Rysunek 3.2: Schemat działania Merge Sort.

Źródło: https://www.tutorialspoint.com/cach3.com/parallel_algorithm/parallel_algorithm_quick_guide.htm.

3.3 Parallel Bucket Sort

Bucket Sort jest algorytmem sortowania, którego działanie opiera się na podziale zakresu wartości danych wejściowych na określoną liczbę kubelków (ang. *buckets*). Każdy element wejściowy jest przypisywany do odpowiedniego kubelka na podstawie swojej wartości, a następnie zawartość poszczególnych kubelków jest sortowana niezależnie. Na końcu posortowane kubelki są łączone w ustalonej kolejności, tworząc wynikowy ciąg [5].

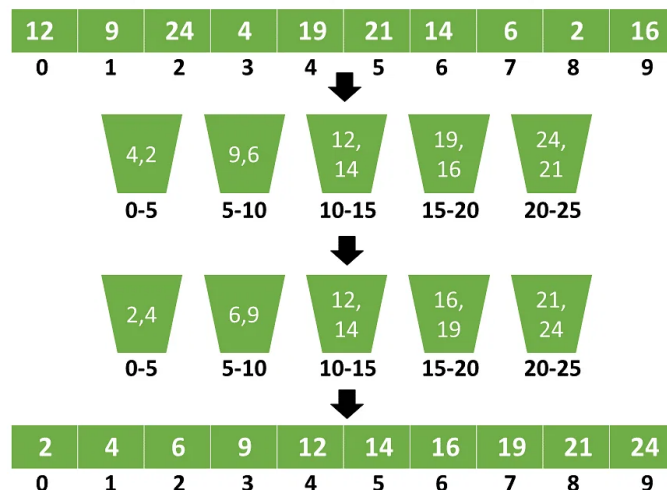
Przy założeniu odpowiedniego rozkładu danych wejściowych Bucket Sort może osiągać średnią złożoność czasową $O(n)$. W klasycznej analizie zakłada się równomierny i niezależny rozkład elementów, dzięki czemu można oczekiwać, że liczba elementów przypadających na poszczególne kubelki będzie zbliżona. Każdy kubek może być następnie sortowany za pomocą innego algorytmu, na przykład Insertion Sort, a ostateczny wynik powstaje przez konkatenację kubelków w kolejności [5].

W wersji równoległej możliwe jest zrównoleglenie zarówno etapu rozmieszczania elementów w kubelkach, jak i sortowania ich zawartości. Dane wejściowe mogą zostać podzielone na części, które są przetwarzane równolegle przez niezależne wątki. Po zakończeniu rozmieszczania elementów kubelki mogą zostać podzielone na grupy, których sortowanie również odbywa się równolegle [6]. Po zakończeniu tej fazy posortowane kubelki są łączone w odpowiedniej kolejności w jeden uporządkowany ciąg.

Efektywność Parallel Bucket Sort jest w dużym stopniu zależna od sposobu rozłożenia elementów pomiędzy kubelkami. Przy równomiernym wypełnieniu kubelków możliwe jest bardziej równomierne rozłożenie pracy pomiędzy jednostki wykonawcze. Jeżeli natomiast część kubelków zawiera znacznie więcej elementów niż pozostałe, może prowadzić to do nierównomiernego obciążenia i ograniczenia uzyskiwanego przyspieszenia. Dodatkowym ograniczeniem jest narzut związany z realizacją obliczeń równoległych, w tym z synchronizacją oraz organizacją pracy pomiędzy jednostkami wykonawczymi. Dla małych zbiorów danych koszt ten może przewyższać korzyści wynikające ze zrównoleglenia, natomiast dla większych zbiorów możliwe jest uzyskanie lepszej wydajności [6].

3.4 Parallel Samplesort

Samplesort jest algorytmem sortowania wykorzystującym podział danych na wiele mniejszych kubelków na podstawie odpowiednio dobranych elementów rozdzielających. Dla większych zbiorów danych z ciągu wejściowego wybierana jest próbka (ang. *sample*), która jest następnie sortowana. Na jej podstawie wyznaczane są elementy rozdzielające (ang. *splitters*), określające granice poszczególnych kubelków.



Rysunek 3.3: Schemat działania Bucket Sort.

Źródło: <https://guideofgreece.com/>.

Elementy wejściowe są przypisywane do odpowiednich kubeków zgodnie z wartościami splitterów, po czym zawartość kubeków jest sortowana, a wyniki są łączone w jeden uporządkowany ciąg [2, 7].

Samplesort można traktować jako uogólnienie Quicksortu wykorzystujące jednocześnie wiele elementów rozdzielających. Zamiast pojedynczego pivota stosuje się $k - 1$ splitterów, które umożliwiają podział danych na k kubeków. Splittery mogą być wyznaczane na podstawie posortowanej próbki poprzez wybór elementów rozmieszczonych w niej w regularnych odstępach. Odpowiedni dobór próbki i splitterów pozwala uzyskać kubki o zbliżonych rozmiarach, co sprzyja bardziej równomiernemu podziałowi pracy [7].

W wersji równoległej dane wejściowe mogą zostać podzielone pomiędzy niezależne jednostki wykonawcze. Każda z nich sortuje lokalnie przypisany fragment danych oraz wybiera z niego elementy próbki. Na podstawie zebranych próbek wyznaczane są globalne elementy rozdzielające, które następnie wykorzystywane są do podziału lokalnych danych na odpowiednie części. Powstałe części są przypisywane jednostkom wykonawczym odpowiedzialnym za ich dalsze przetwarzanie. Po zakończeniu tej fazy uporządkowane fragmenty tworzą wynikowy posortowany ciąg [8].

Istotną zaletą Samplesort jest możliwość wyznaczania granic podziału na podstawie informacji uzyskanych z próbki danych. Odpowiednio dobrane elementy rozdzielające pozwalają ograniczyć różnice pomiędzy rozmiarami powstających części, a tym samym sprzyjają równomiernemu rozłożeniu pracy pomiędzy jednostki wykonawcze [7, 8]. Dodatkowy koszt algorytmu wynika natomiast z konieczności wyboru i sortowania próbki, wyznaczenia splitterów oraz dystrybucji elementów pomiędzy utworzone części. Dla odpowiednio małych fragmentów danych dalsze sortowanie

może być realizowane za pomocą algorytmu sekwencyjnego [7].

Initial data (after randomizing)



Choose splitters (2 and 6)



Sort local data



Starch



Rysunek 3.4: Schemat działania Samplesort.

Źródło: opracowanie własne na podstawie [8].

Rozdział 4

Środowisko badawcze i implementacja

4.1 Opis środowiska

Wszystkie eksperymenty zostały przeprowadzone na komputerze o następującej konfiguracji:

- ▶ **Procesor:** Intel(R) Core(TM) i9-9980XE CPU @ 3.00 GHz
- ▶ **Rdzenie fizyczne:** 18
- ▶ **Rdzenie logiczne:** 36
- ▶ **Taktowanie bazowe CPU:** 3000 MHz
- ▶ **Pamięć operacyjna RAM:** 127,68 GB
- ▶ **System operacyjny:** Windows Server 2019
- ▶ **Architektura:** AMD64 (x86-64)
- ▶ **Interpreter Python:** 3.14.7

Środowisko to umożliwia przeprowadzenie badań algorytmów sortowania równoległego dla różnych poziomów równoległości. Procesor wyposażony w 18 rdzeni fizycznych i 36 procesorów logicznych pozwala na analizę zmian wydajności wraz ze wzrostem liczby wykorzystywanych jednostek wykonawczych.

Dane dotyczące sprzętu i oprogramowania były automatycznie zebrane i zapisywane w bazie danych SQLite przed przeprowadzaniem eksperymentów.

4.2 Wybór języka i bibliotek

Do implementacji algorytmów sortowania równoległego, genrowania danych, przeprowadzenia eksperymentów oraz wizualizacji wyników wybrano język programowania Python w wersji 3.11.7 [14]. Decyzja ta wynikała z kilku powodów. Python jest jednym z najpopularniejszych języków, charakteryzuje się przejrzystą i elegancką składnią, co ułatwia tworzenie, analizę i utrzymanie kodu. Posiada bogatą bibliotekę standardową oraz rozbudowany zbiór dostępnych bibliotek, pakietów i narzędzi programistycznych rozszerzających możliwości języka, które znacząco przyspieszają rozwój oprogramowania. Dodatkowo język ten jest w pełni przenośny między systemami operacyjnymi tj. Windows, Linux, macOS, co ma znaczenie przy planowaniu ewentualnych dalszych testów na innych platformach.

Niezwyczajnie istotną rolę w projekcie odgrywa biblioteka `multiprocessing` [15], stanowiąca część standardowej biblioteki Pythona. Pakiet ten przy użyciu interfejsu programistycznego podobnego do pythonowego modułu `threading` umożliwia tworzenie procesów oraz realizację obliczeń równoległych z pominięciem ograniczeń wynikających z Global Interpreter Lock (GIL). Dzięki temu możliwe jest efektywne wykorzystanie wielu rdzeni procesora. W ramach tej biblioteki wykorzystano również moduł `multiprocessing.shared_memory` [16], który pozwala na bezpośredni dostęp wielu procesów do wspólnego obszaru pamięci. Mechanizm ten eliminuje konieczność serializacji oraz przesyłania dużych struktur danych między procesami, ograniczając związany z tym narzut, co ma szczególne znaczenie podczas sortowania dużych zbiorów danych.

Biblioteka `NumPy` [17] obsługująca dane numeryczne i szybkie operacje na tablicach, została wykorzystana w optymalizacji algorytmów Parallel Bucket Sort oraz Parallel Samplesort. Posłużyła w nich do efektywnego rozdzielania elementów do kubeków, co znacznie przyspieszyło etap dystrybucji danych. Ponadto `NumPy` wykorzystano w module analizy złożoności obliczeniowej oraz przy generowaniu wykresów i rankingów.

Wyniki eksperymentów były przechowywane i przetwarzane z użyciem biblioteki `pandas` [18], która oferuje wygodne struktury danych (`DataFrame`) oraz narzędzia do filtrowania, agregacji i analizy tabelarycznej. Do generowania wykresów i wizualizacji wyników wykorzystano bibliotekę `matplotlib` [19].

Do monitorowania wykorzystanych zasobów systemowych wykorzystano bibliotekę `psutil` [20] oraz narzędzia `py-cpuinfo` i `py-spy` [22].

Całość kodu źródłowego została opracowana w środowisku programistycznym JetBrains PyCharm [23], które zapewnia wsparcie dla języka Python, debugowanie oraz wygodne zarządzanie projektem.

Do przechowywania wygenerowanych danych testowych oraz wyników eksperymentów wykorzystano bazę danych SQLite [24]. Jest to wbudowany silnik bazo-

danowy, który nie wymaga osobnego serwera ponieważ odczytuje i zapisuje dane bezpośrednio do plików dyskowych. Rozwiązanie to upraszcza zarządzanie danymi i zapewnia łatwą przenośność między środowiskami.

Wybór powyższego zestawu technologii umożliwia na spójną implementację algorytmów, precyzyjny pomiar ich charakterystyk wydajnościowych oraz wygodną analizę i prezentację wyników badań.

4.3 Implementacja wybranych algorytmów

Wszystkie algorytmy zaimplementowano w języku Python z wykorzystaniem mechanizmu wieloprotocowego oraz pamięci współdzielonej. Każdy algorytm występuje w dwóch wariantach: sekwencyjnym, który stanowi punkt odniesienia oraz równoległym. Wspólną cechą implementacji równoległych jest przechowywanie sortowanych danych w bloku pamięci współdzielonej, mapowanym na tablicę typów `ctypes` (`c_longlong` dla liczb całkowitych oraz `c_double` dla liczb zmiennoprzecinkowych). Dzięki temu procesy potomne mogą odczytywać i modyfikować te same dane bez konieczności ich kopiowania oraz przesyłania pomiędzy procesami.

Ogólna architektura rozwiązania

Przy algorytmach opartych na strategii “dziel i zwyciężaj” czyli Parallel Quicksort oraz Parallel Merge Sort, zastosowano rekurencyjne tworzenie procesów z ograniczeniem głębokości równoległości za pomocą parametru `max_depth`. Dodatkowo wprowadzono progi minimalnego rozmiaru podproblemu zawarte w słowniku `PARALLEL_CUTOFF`, poniżej których dalsze tworzenie procesów jest nieopłacalne i następuje przejście do sortowania sekwencyjnego. Takie rozwiązanie ogranicza narzut związany z tworzeniem zbyt dużej liczby procesów przy małych fragmentach danych.

Dla algorytmów opartych na kubekach czyli Parallel Bucket Sort oraz Parallel Samplesort, zastosowano model, w którym liczba procesów jest określana na podstawie liczby rdzeni wykorzystywanych w danym eksperymencie. Dane są najpierw rozdzielane do kubków, a następnie grupy kubków są przydzielane poszczególnym procesom. Również tutaj wprowadzono progi minimalnego rozmiaru zawarte w słownikach `PARALLEL_SIZE_CUTOFF` oraz `GROUP_SIZE_CUTOFF`, poniżej których algorytm przełącza się na wariant sekwencyjny.

Parallel Quicksort

Wariant sekwencyjny realizuje klasyczny Quicksort z wyborem pivota jako elementu środkowego.

```

1 def partition(arr, low, high):
2     pivot = arr[(low + high) // 2]
3
4     i = low
5     j = high
6
7     while i <= j:
8         while arr[i] < pivot:
9             i += 1
10        while arr[j] > pivot:
11            j -= 1
12        if i <= j:
13            arr[i], arr[j] = arr[j], arr[i]
14            i += 1
15            j -= 1
16
17    return i

```

Listing 4.1: Funkcja partycjonowania w Quicksort

W wersji równoległej po wykonaniu partycji lewa i prawa część tablicy mogą być sortowane niezależnie przez osobne procesy. Każdy proces dołącza się do istniejącego bloku pamięci współdzielonej, sortuje przypisany dla siebie zakres, a następnie kończy działanie. Gdy rozmiar zakresu spada poniżej ustalonego progu lub osiągnięta zostaje maksymalna głębokość równoległości, dalsze sortowanie odbywa się sekwencyjnie.

```

1 if size <= min_size or depth >= max_depth:
2     sort_in_place_on_shared(arr, low, high)
3     return
4
5 local = arr[low:high + 1]
6 local_index = partition(local, 0, size - 1)
7 arr[low:high + 1] = local
8 index = low + local_index
9
10 left = (low, index - 1)
11 right = (index, high)
12
13 processes = []
14
15 for part in [left, right]:
16     p_low, p_high = part
17     if p_low >= p_high:
18         continue
19
20     part_size = p_high - p_low + 1
21     if part_size > min_size:

```

```

22         p = mp.Process(
23             target=parallel_quicksort_worker,
24             args=(shm_name, length, dtype, p_low, p_high,
25                 depth + 1, max_depth, min_size)
26         )
27         p.start()
28         processes.append(p)
29     else:
30         sort_in_place_on_shared(arr, p_low, p_high)
31
32 for p in processes:
33     p.join()

```

Listing 4.2: Tworzenie procesów w Parallel Quicksort

Parallel Merge Sort

Wariant sekwencyjny realizuje klasyczny Merge Sort z rekurencyjnym podziałem tablicy na połowy oraz scalaniem posortowanych części.

```

1 def merge(arr, left, mid, right):
2     temp = [None] * (right - left + 1)
3
4     i = left
5     j = mid + 1
6     k = 0
7
8     while i <= mid and j <= right:
9         if arr[i] <= arr[j]:
10             temp[k] = arr[i]
11             i += 1
12         else:
13             temp[k] = arr[j]
14             j += 1
15
16         k += 1
17
18     while i <= mid:
19         temp[k] = arr[i]
20         i += 1
21         k += 1
22
23     while j <= right:
24         temp[k] = arr[j]
25         j += 1
26         k += 1
27
28     arr[left:right + 1] = temp

```

Listing 4.3: Funkcja scalania w Merge Sort

W wersji równoległej obie połowy mogą być sortowane jednocześnie przez osobne procesy. Po zakończeniu obu procesów następuje etap scalania, który w danej implementacji wykonywany jest sekwencyjnie. Głębokość równoległości oraz minimalny rozmiar zakresu ograniczają nadmierne tworzenie procesów.

```
1 if size <= min_size or depth >= max_depth:
2     sort_in_place_on_shared(arr, left, right)
3     return
4
5 mid = (left + right) // 2
6 processes = []
7
8 for part_left, part_right in [(left, mid), (mid + 1, right)]:
9     part_size = part_right - part_left + 1
10    if part_size > min_size:
11        p = mp.Process(
12            target=parallel_mergesort_worker,
13            args=(shm_name, length, dtype, part_left, part_right,
14                depth + 1, max_depth, min_size)
15        )
16        p.start()
17        processes.append(p)
18    else:
19        sort_in_place_on_shared(arr, part_left, part_right)
20
21 for process in processes:
22     process.join()
23
24 local = arr[left:right + 1]
25 merge(local, 0, mid - left, size - 1)
26 arr[left:right + 1] = local
```

Listing 4.4: Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort

Parallel Bucket Sort

Wariant sekwencyjny rozdziela elementy do liczby kubełków wyznaczonej na podstawie rozmiaru danych, a następnie sortuje każdy kubełek z wykorzystaniem zaimplementowanego algorytmu Merge Sort. Na sam koniec łączy wyniki w jedną posortowaną tablicę.

```
1 def distribute_to_buckets(arr, bucket_count):
2     if len(arr) == 0:
```

```

3         return []
4
5     arr_np = np.asarray(arr)
6     min_value = arr_np.min()
7     max_value = arr_np.max()
8
9     if min_value == max_value:
10         buckets = [[] for _ in range(bucket_count)]
11         buckets[0] = list(arr)
12         return buckets
13
14     bucket_range = (max_value - min_value) / bucket_count
15     indices = np.minimum(
16         bucket_count - 1,
17         ((arr_np - min_value) / bucket_range).astype(np.int64)
18     )
19
20     order = np.argsort(indices, kind='stable')
21     sorted_indices = indices[order]
22     sorted_values = arr_np[order]
23
24     buckets = [[] for _ in range(bucket_count)]
25     boundaries = np.searchsorted(sorted_indices, np.arange(bucket_count +
26         1))
27     for i in range(bucket_count):
28         buckets[i] = sorted_values[boundaries[i]:boundaries[i + 1]].
29         tolist()
30
31     return buckets

```

Listing 4.5: Rozdzielanie elementów do kubeków w Bucket Sort

W wersji równoległej liczba utworzonych kubeków jest wyliczana na podstawie rozmiaru danych oraz liczby wykorzystywanych rdzeni. Po utworzeniu kubeków ich zawartość jest łączona w jedną tablicę, która umieszczana zostaje w pamięci współdzielonej, a zakresy poszczególnych kubeków są grupowane i przydzielane procesom. Procesy sortują przypisane im grupy kubeków niezależnie. Dla małych grup sortowanie wykonywane jest sekwencyjnie, co ogranicza narzut związany z tworzeniem procesów. Do efektywnego rozdzielania elementów wykorzystano operacje wektorowe biblioteki NumPy.

```

1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4
5 for group in bucket_groups:
6     if not group:
7         continue

```

```

8
9     group_size = group[-1][1] - group[0][0] + 1
10
11     if should_spawn_for_group(group_size, process_count):
12         process = mp.Process(
13             target=bucket_worker,
14             args=(shm.name, len(arr), dtype, group)
15         )
16         process.start()
17         processes.append(process)
18     else:
19         sequential_groups.append(group)
20
21 for group in sequential_groups:
22     bucket_worker_inline(arr, group)
23
24 for process in processes:
25     process.join()

```

Listing 4.6: Przydzielanie grup kubełków procesom w Parallel Bucket Sort

Parallel Samplesort

Wariant sekwencyjny działa w kilku etapach: podział danych na części, lokalne sortowanie, wybór próbek, wyznaczenie wartości rozdzielających, rozdzielenie elementów do kubełków według wyznaczonych wartości oraz ostateczne sortowanie kubełków.

```

1 def distribute_to_buckets(data, pivots):
2     if not pivots:
3         return [list(data)]
4
5     arr = np.asarray(data)
6     pivots_arr = np.asarray(pivots)
7
8     indices = np.searchsorted(pivots_arr, arr, side='right')
9
10    order = np.argsort(indices, kind='stable')
11    sorted_indices = indices[order]
12    sorted_values = arr[order]
13    boundaries = np.searchsorted(sorted_indices, np.arange(len(pivots) +
14        2))
15
16    buckets = []
17    for i in range(len(pivots) + 1):
18        buckets.append(sorted_values[boundaries[i]:boundaries[i + 1]].
19            tolist())

```

```
19         return buckets
```

Listing 4.7: Rozdzielanie elementów do kubków na podstawie wartości rozdzielających w Samplesort

W wersji równoległej etapy te realizowane są z wykorzystaniem wielu procesów. Na początku procesy równoległe sortują lokalne fragmenty danych i zbierają próbki. Na podstawie próbek wyznaczane są wartości rozdzielające, po czym dane elementy są rozdzielane do kubków. Na końcu grupy kubków są ponownie sortowane równoległe. W przypadku niewielkich grup danych stosowany jest wariant sekwencyjny, co ogranicza koszt związany z tworzeniem procesów. Do dystrybucji elementów wykorzystano operacje wektorowe biblioteki NumPy.

```
1 bucket_groups = split_bucket_ranges(bucket_ranges, process_count)
2 processes = []
3 sequential_groups = []
4 min_group_size = get_group_size_cutoff(process_count)
5
6 for group in bucket_groups:
7     if not group:
8         continue
9
10    group_size = group[-1][1] - group[0][0] + 1
11
12    if group_size > min_group_size:
13        process = mp.Process(
14            target=bucket_worker,
15            args=(shm.name, len(arr), dtype, group)
16        )
17        process.start()
18        processes.append(process)
19    else:
20        sequential_groups.append(group)
21
22 for group in sequential_groups:
23     sort_group(arr, group)
24
25 for process in processes:
26     process.join()
```

Listing 4.8: Równoległe sortowanie grup kubków w Parallel Samplesort

4.4 Charakterystyka danych testowych

Do przeprowadzenia testów przygotowano zestawy danych o zróżnicowanej charakterystyce, tak aby możliwe było ocenienie zachowania algorytmów w różnych scenariuszach.

Typy i rozmiary danych

Testy przeprowadzono dla dwóch typów wartości:

- ▶ liczb całkowitych - `int`
- ▶ liczb zmiennoprzecinkowych - `float`

Dla każdego typu wygenerowano zbiory o następujących rozmiarach:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów
- ▶ 1 000 000 elementów

Wygenerowane wartości znajdują się w zakresie od 0 do 1 000 000.

Charakterystyka zbiorów

Przygotowano trzy główne kategorie danych:

- ▶ **Dane losowe** – elementy generowane w sposób całkowicie losowy z użyciem generatora liczb pseudolosowych.
- ▶ **Dane z duplikatami** – zbiory, w których około 40% elementów stanowią powtórzenia. Najpierw wygenerowano 60% unikalnych wartości zbioru o danym rozmiarze, a następnie na ich podstawie dodano brakującą część poprzez losowe powtórzenia.
- ▶ **Dane częściowo posortowane** – zbiory zawierające odpowiednio 20%, 40%, 60% oraz 80% elementów uporządkowanych. Posortowane fragmenty przeplatano fragmentami losowymi, co pozwoliło uzyskać zbiory o określonym stopniu częściowego uporządkowania.

```
1 def generate_part_sorted_values(table_name, count, scope, sorted_ratio):
2     sorted_count = int(count * sorted_ratio)
3     random_count = count - sorted_count
4     parts = 5
5
6     if "int" in table_name:
7         sorted_values = sorted(random.randint(0, scope) for _ in range(
8             sorted_count))
9         random_values = [random.randint(0, scope) for _ in range(
10             random_count)]
11     else:
```



```

10     sorted_values = sorted(random.uniform(0, scope) for _ in range(
        sorted_count))
11     random_values = [random.uniform(0, scope) for _ in range(
        random_count)]
12
13     chunk_size = sorted_count // parts
14     sorted_chunks = [sorted_values[i*chunk_size:(i+1)*chunk_size] for i
        in range(parts-1)]
15     sorted_chunks.append(sorted_values[(parts-1)*chunk_size:])
16
17     random_chunk_size = random_count // (parts + 1)
18     random_chunks = [random_values[i*random_chunk_size:(i+1)*
        random_chunk_size] for i in range(parts)]
19     random_chunks.append(random_values[parts*random_chunk_size:])
20
21     combined = []
22     for i in range(parts):
23         combined.extend(random_chunks[i])
24         combined.extend(sorted_chunks[i])
25     combined.extend(random_chunks[-1])
26
27     values = [(val, count) for val in combined]
28
29     return values

```

Listing 4.9: Generowanie danych częściowo posortowanych

Do testów przygotowano dwanaście wariantów zbiorów (2 typy danych X 6 charakterystyk).

Sposób przechowywania

Wszystkie dane testowe zapisano w bazie danych SQLite. Każdy wariant przechowywany był w osobnej tabeli o strukturze:

- ▶ `id` – unikalny identyfikator rekordu,
- ▶ `value` – wartość elementu `INTEGER` lub `REAL`
- ▶ `set_size` – rozmiar zbioru, do którego należy dany element, ograniczony do wartości 1000, 10 000, 100 000, 1000 000

Rozdział 5

Metodyka badań

5.1 Scenariusze testowe

Eksperymenty przeprowadzono według różnych konfiguracji parametrów, których celem jest zebranie informacji potrzebnych do przeprowadzenia analizy.

Zakres badanych konfiguracji

W badaniach uwzględniono cztery algorytmy:

- ▶ Quick Sort
- ▶ Merge Sort
- ▶ Bucket Sort
- ▶ Sample Sort

Dla każdego algorytmu wykonano pomiary jego wariantu sekwencyjnego oraz wersję równoległą. Wariant sekwencyjny stanowił punkt odniesienia i był zawsze uruchamiany na jednym rdzeniu.

Testy przeprowadzono dla następujących rozmiarów danych:

- ▶ 1 000 elementów
- ▶ 10 000 elementów
- ▶ 100 000 elementów
- ▶ 1000 000 elementów

Wykorzystano dwanaście wariantów zbiorów wejściowych, obejmujących:

- ▶ dane losowe
- ▶ dane z duplikatami

- ▶ dane częściowo posortowane (20%, 40%, 60% oraz 80% elementów uporządkowanych)

zarówno dla liczb całkowitych, jak i zmiennoprzecinkowych.

Konfiguracja równoległości

Testy równoległe wykonano dla dostępnych konfiguracji liczby rdzeni udostępnianych przez środowisko sprzętowe. W przypadku algorytmów Quick Sort oraz Merge Sort liczba procesów była zależna od parametru głębokości równoległości:

$$max_depth = \log_2(cores).$$

Dla Bucket Sort oraz Samplesort liczbę procesów ustawiano jako liczbę wykorzystanych rdzeni.

Organizacja eksperymentów

Każdy pomiar wykonywano wielokrotnie, a do analizy przyjmowano wartości uśrednione. Liczbę powtórzeń ustalono na 10/100 dla każdego wariantu eksperymentu, aby uzyskać możliwie dokładną wartość średnią z przeprowadzonych pomiarów. Przed właściwymi pomiarami wykonano dodatkowe uruchomienie, którego wynik nie był uwzględniany w obliczeniach. Miało ono na celu przygotowanie środowiska do właściwych testów, jak również zprofilowanie uruchomionego algorytmu, dzięki czemu narzut pracy profilera nie wpływał na wyniki.

Przeprowadzone testy pozwoliły uzyskać zróżnicowany zestaw wyników, które umożliwiają analizę wpływu rozmiaru danych, liczby wykorzystywanych rdzeni oraz charakterystyki zbioru wejściowego na czas sortowania, zużycie pamięci oraz efektywność zrównoleglenia.

5.2 Wariant sekwencyjny jako punkt odniesienia

Każdy z badanych algorytmów, oprócz wersji równoległej posiada również swój wariant sekwencyjny. Stanowił on punkt odniesienia dla przeprowadzanych po nim testów wersji równoległej, umożliwiając określenie, jak wariant równoległy wypada na tle odpowiadającego mu wariantu sekwencyjnego.

Uzyskane wyniki z wersji sekwencyjnej wykorzystywano następnie do wyznaczenia metryk efektywności zrównoleglenia:

- ▶ przyspieszenia (*speedup*)
- ▶ efektywności (*efficiency*)

Wariant sekwencyjny jest wykonywany na pojedynczym rdzeniu, z wykorzystaniem identycznych danych wejściowych oraz tych samych metod pomiaru czasu sortowania, zużycia pamięci RAM i obciążenia procesora co w odpowiadającym mu wariantcie równoległym. Jednakowe warunki umożliwiają skupienie analizy na rzeczywistych różnicach pomiędzy wariantami i uzyskanymi przez nich wynikami.

5.3 Parametry pomiarów

Podczas każdego eksperymentu zbierane są wyniki pozwalające na późniejszą analizę wydajności oraz efektywności algorytmów oraz wykorzystania zasobów sprzętowych.

Podstawowym parametrem jest czas wykonania algorytmu, mierzony z wysoką rozdzielczością za pomocą funkcji `time.perf_counter`. Dla serii powtórzeń zapisywano:

- ▶ czas średni
- ▶ medianę
- ▶ odchylenie standardowe

Dodatkowo monitorowano zużycie zasobów systemowych:

- ▶ średnie obciążenie procesora CPU w trakcie działania algorytmu
- ▶ średnie zużycie pamięci operacyjnej RAM
- ▶ maksymalne zużycie pamięci operacyjnej RAM

Na podstawie czasu sekwencyjnego T_1 oraz czasu równoległego T_p wyznaczano metryki efektywności zrównoleglenia:

- ▶ przyspieszenie (*speedup*):

$$S_p = \frac{T_1}{T_p},$$

- ▶ efektywność (*efficiency*):

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p},$$

gdzie p oznacza liczbę wykorzystanych rdzeni.

Wartości *speedup* oraz *efficiency* dla wariantu sekwencyjnego są pomijane

Wszystkie uzyskane wyniki zapisywano w bazie danych SQLite, co umożliwiło ich bezpieczne przechowywanie oraz wykorzystanie w dalszych etapach badań.

5.4 Narzędzia do pomiaru czasu, CPU i pamięci

Do realizacji potrzebnych pomiarów wykorzystano zestaw narzędzi i bibliotek umożliwiających precyzyjne rejestrowanie czasu wykonania oraz monitorowanie wykorzystania zasobów systemowych w trakcie działania algorytmów.

Pomiar czasu

Zapewnia ona dostęp do zegara o wysokiej rozdzielczości, przeznaczonego do pomiaru krótkich przedziałów czasu. Czas wykonania wyznaczany jest jako różnica wartości licznika zarejestrowanych przed rozpoczęciem i po zakończeniu badanego fragmentu kodu. Dzięki wysokiej rozdzielczości funkcja dobrze nadaje się do przeprowadzania pomiarów wydajnościowych. W implementacji CPython wykorzystywany zegar jest monotoniczny, dzięki czemu jego wartość nie może cofać się w czasie.

Monitorowanie CPU i pamięci RAM

Zużycie procesora oraz pamięci operacyjnej monitorowano z wykorzystaniem biblioteki `psutil` [20]. Pomiary realizowano w osobnym wątku, który w ustalonych odstępach czasu zapisanych w `sample_interval` odczytywał informacje o wykorzystanych zasobach. Dla zbiorów danych o rozmiarze nieprzekraczającym 10 000 odstęp wynosił 0,01 sekundy, natomiast dla pozostałych zbiorów 0,05 sekundy. Podczas każdego pomiaru odczytano:

- ▶ łączne obciążenie CPU procesu głównego oraz procesów potomnych
- ▶ zużycie pamięci RAM na podstawie wartości RSS

Dzięki uwzględnieniu procesów potomnych możliwe było poprawne oszacowanie wykorzystywanych zasobów przez algorytmy równoległe.

Profilowanie wydajności

Do analizy szczegółowej wydajności wykorzystano dwa narzędzia:

- ▶ `cProfile` [21] – wbudowany profiler Pythona, umożliwiający pomiar liczby wywołań oraz czasu wykonania poszczególnych funkcji
- ▶ `py-spy` [22] – zewnętrzny profiler próbkujący umożliwiający analizę działania procesu głównego i podprocesów oraz generowanie typu *flamegraph*

Profilowanie nie wpływało na wartości raportowane w głównych wynikach, ponieważ pierwsze uruchomienie, na którym działały profilery było pomijane przy obliczaniu statystyk.

Organizacja pomiaru

Każdy pojedynczy przebieg algorytmu uruchamiano w osobnym procesie roboczym, co zapewniało izolację pomiaru oraz zapewniało zabezpieczenie testów w przypadku awarii lub zawieszeniu badanego algorytmu. Wprowadzono limit czasowy ustawiony na 15 minut, po przekroczeniu którego proces był przerywany, a dany test otrzymywał status `TIMEOUT`.

Zebrane próbki CPU i RAM agregowano do wartości średnich oraz maksymalnych, a następnie wraz z metrykami czasu, *speedup* i *efficiency* zapisano w bazie danych.

5.5 Przebieg eksperymentów

Opis jak zostały przeprowadzone testy. Przygotowanie środowisk testowych. Sposób zapisywania i archiwizacji wyników. Procedura uruchomienia algorytmu itp.

Dopytać czy ta sekcja w ogóle ma sens !!!!!!!!!!!!!

Rozdział 6

Analiza wyników badań

Celem niniejszego rozdziału jest przedstawienie oraz przeanalizowanie wyników przeprowadzonych eksperymentów. Zaimplementowane algorytmy przeanalizowano pod względem czasu sortowania, uzyskanego przyspieszenia i efektywności, wykorzystania zasobów oraz skalowalności. Poza tym uwzględniono również wpływ rozmiaru, typu i charakterystyki danych wejściowych.

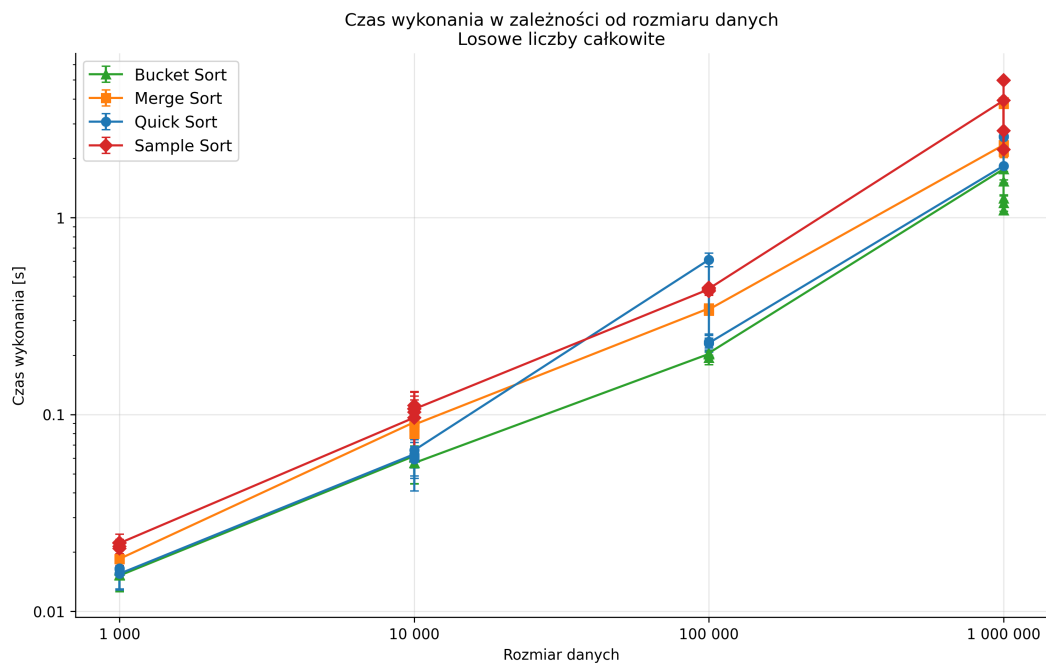
6.1 Porównanie czasu sortowania

Pierwszym i podstawowym kryterium wykorzystanym do oceny wydajności badanych algorytmów był czas wykonania sortowania. Analizie poddano warianty sekwencyjne oraz równoległe algorytmów Quick Sort, Merge Sort, Bucket Sort i Sample Sort dla zbiorów danych o rozmiarach 1 000, 10 000, 100 000 oraz 1 000 000 elementów.

W pierwszej kolejności przeanalizowano wpływ rozmiaru danych na czas wykonania poszczególnych algorytmów. Jako punkt odniesienia przyjęto zbiór danych `random_int`, zawierający losowo wygenerowane liczby całkowite. Pozwala to na porównanie badanych algorytmów w jednakowych warunkach, bez uwzględniania na tym etapie wpływu stopnia uporządkowania danych lub występowania duplikatów.

Dla wszystkich badanych algorytmów zwiększenie rozmiaru zbioru skutkowało wzrostem czasu wykonania, przy czym różnice dla różnych algorytmów szczególnie stawały się widoczne dla największego badanego zbioru.

Zestaw danych obejmujący 1 000 000 losowych liczb całkowitych uzyskał najkrótszy średni czas wykonania dla algorytmu Bucket Sort, osiągając około 2,04 s. Nieco dłuższego czasu wymagał Quick Sort z czasem około 2,25 s. Merge Sort wymagał około 3,41 s, natomiast Sample Sort osiągnął najdłuższy czas spośród porównywanych algorytmów, równy 4,95 s. Wyniki wskazują, że różnice pomiędzy badanymi implementacjami zwiększają się wraz ze wzrostem rozmiaru przetwarzanego zbioru.



Rysunek 6.1: Średni czas wykonania wariantów sekwencyjnych w zależności od rozmiaru zbioru `random_int`.

Źródło: opracowanie własne.

W celu bezpośredniego porównania wariantów sekwencyjnych i równoległych w tabeli 6.1 zestawiono czasy uzyskane dla 1 000 000 losowych liczb całkowitych. Dla każdego algorytmu przedstawiono czas wariantu sekwencyjnego oraz najlepszy czas uzyskany spośród badanych konfiguracji równoległych.

Algorytm	Czas sekw. [s]	Najlepszy równ. [s]	Jednostki	Speedup
Quick Sort	2.2484	1.8304	2	1.2284
Merge Sort	3.4149	2.1621	4	1.5795
Bucket Sort	2.0409	1.0903	16	1.8718
Sample Sort	4.9488	2.2156	8	2.2336

Tabela 6.1: Najlepsze czasy wariantów równoległych w porównaniu z wariantami sekwencyjnymi dla 1 000 000 elementów zbioru `random_int`.

Zestawienie pokazuje, że dla każdego z badanych algorytmów możliwe było uzyskanie czasu krótszego niż w wariacie sekwencyjnym, niemniej najlepsze wyniki osiągnęto przy różnej liczbie jednostek wykonawczych. Największe przyspieszenie w stosunku do wykonania sekwencyjnego odnotowano dla Sample Sort i wyniosło ono około 2,23. Nie oznaczało to jednak najkrótszego bezwzględnego czasu sortowania. Najlepszy wynik czasowy uzyskał Bucket Sort, dla którego przy wykorzystaniu 16 jednostek wykonawczych wyniósł on około 1,09 s.

Uzyskane wyniki wskazują, że zastosowanie wariantu równoległego może pro-

wadzić do skrócenia czasu sortowania, jednak wielkość uzyskanej poprawy jest zależna od badanego algorytmu oraz zastosowanej konfiguracji równoległości.

6.2 Wpływ liczby jednostek wykonawczych na wydajność

Kolejnym analizowanym aspektem był wpływ liczby wykorzystywanych jednostek wykonawczych na czas działania równoległych wariantów badanych algorytmów. Analizę przeprowadzono dla zbioru `random_int` zawierającego 1 000 000 elementów, wykorzystując kolejno 2, 4, 8, 16 oraz 32 jednostki wykonawcze. Pozwoliło to przeanalizować, jak zmiana liczby jednostek wykonawczych wpływa na czas sortowania.

W celu porównania wpływu liczby jednostek wykonawczych na wydajność poszczególnych algorytmów zestawiono wyniki uzyskane dla wszystkich badanych konfiguracji.

Rdzenie	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
2	1.8304	2.3365	1.7634	3.9321
4	2.2213	2.1621	1.5249	2.7569
8	2.5613	2.3212	1.1885	2.2156
16	2.5835	3.7816	1.0903	4.9560
32	2.5879	3.8521	1.2501	4.9891

Tabela 6.2: Czas wykonania badanych algorytmów w zależności od liczby jednostek wykonawczych.

Uzyskane wyniki wskazują wyraźne różnice pomiędzy badanymi algorytmami. Wzrost liczby jednostek wykonawczych nie zawsze przekładał się na poprawę uzyskiwanego czasu sortowania, a najkrótsze czasy osiągnęto przy różnej liczbie wykorzystywanych jednostek.

W przypadku Quick Sort najkrótszy średni czas wykonania uzyskano już przy wykorzystaniu 2 jednostek wykonawczych i wyniósł on około 1,83 s. Zwiększenie ich liczby do 4 skutkowało wydłużeniem czasu do około 2,22 s, natomiast dla 8 jednostek osiągnął on około 2,56 s. Dalsze zwiększanie liczby jednostek do 16 i 32 nie powodowało już wyraźnych różnic w czasie wykonania, który utrzymywał się na poziomie około 2,58 s. Wyniki wskazują, że dla badanej implementacji Quick Sort zwiększenie liczby jednostek wykonawczych powyżej 2 nie przynosiło dalszych korzyści pod względem czasu wykonania.

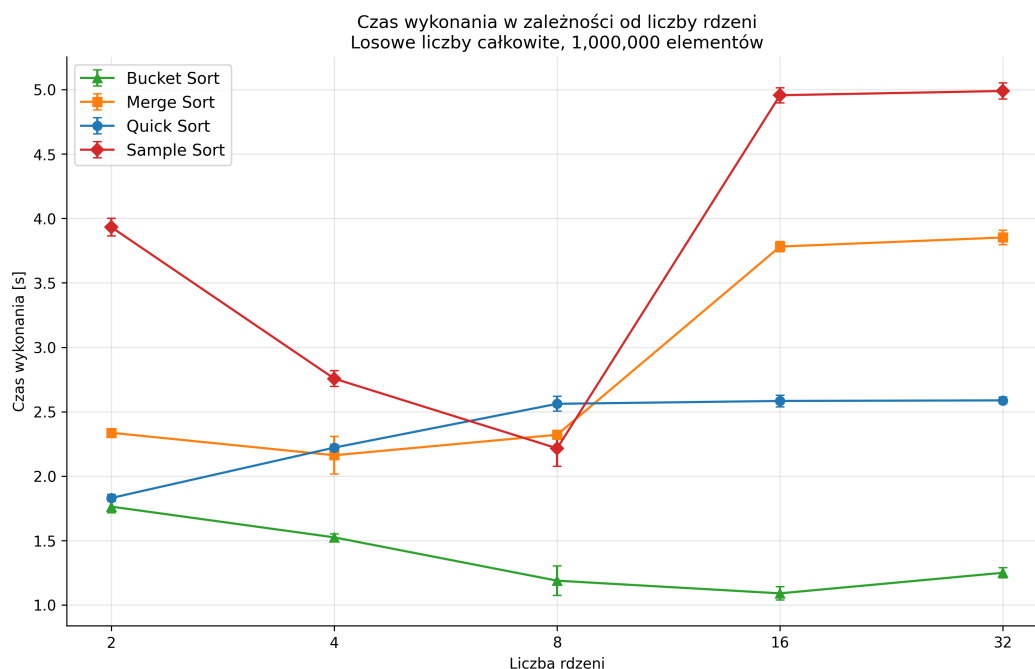
Dla Merge Sort zwiększenie liczby jednostek wykonawczych z 2 do 4 pozwoliło skrócić czas z około 2,34 s do 2,16 s. Dalsze zwiększenie ich liczby do 8 nie

przyniosło dalszej poprawy, a czas wyniósł około 2,32 s. Dla 16 oraz 32 jednostek wykonawczych odnotowano wyraźne wydłużenie czasu wykonania, który wyniósł odpowiednio około 3,78 s oraz 3,85 s. Najkorzystniejszy wynik dla Merge Sort osiągnięto więc przy 4 jednostkach wykonawczych.

Odminną tendencję zaobserwowano dla Bucket Sort. Zwiększanie liczby jednostek wykonawczych od 2 do 16 prowadziło do systematycznego skracania czasu sortowania. Średni czas skrócił się z około 1,76 s dla 2 jednostek do 1,52 s dla 4, 1,19 s dla 8 oraz 1,09 s dla 16 jednostek wykonawczych. Dopiero zwiększenie ich liczby do 32 skutkowało wydłużeniem czasu do około 1,25 s. Spośród badanych algorytmów Bucket Sort wykazywał poprawę czasu wykonania przy zwiększaniu liczby jednostek wykonawczych aż do 16 jednostek.

Sample Sort również początkowo charakteryzował się poprawą wydajności. Średni czas wykonania zmniejszył się z około 3,93 s dla 2 jednostek do 2,76 s dla 4 oraz 2,22 s dla 8 jednostek wykonawczych. Po zwiększeniu ich liczby do 16 nastąpiło jednak wyraźne wydłużenie czasu wykonania do około 4,96 s. Przy 32 jednostkach wartość ta uległa jedynie niewielkiej zmianie osiągając około 4,99 s. Najlepszy wynik uzyskano zatem przy 8 jednostkach wykonawczych.

Zmiany czasu wykonania poszczególnych algorytmów wraz ze zwiększaniem liczby jednostek wykonawczych przedstawiono na rysunku 6.2.



Rysunek 6.2: Średni czas wykonania wariantów równoległych dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.

Źródło: opracowanie własne.

Uzyskane wyniki wskazują, że najkorzystniejsza liczba jednostek wykonawczych różniła się w zależności od badanego algorytmu. Quick Sort osiągnął najlep-

szy wynik przy 2 jednostkach, Merge Sort przy 4, Sample Sort przy 8, natomiast Bucket Sort przy 16 jednostkach wykonawczych. Zwiększanie ich liczby powyżej tych wartości nie prowadziło do dalszej poprawy, a dla niektórych algorytmów wiązało się ze wzrostem czasu wykonania.

W celu określenia, czy podobne zależności występują również dla innego typu danych, przeanalizowano wyniki dla zbioru `random_float`. Dla Merge Sort, Bucket Sort oraz Sample Sort najkrótszy czas wykonania uzyskano przy tej samej liczbie jednostek wykonawczych co dla zbioru `random_int`, odpowiednio przy 4, 16 oraz 8 jednostkach. Wyjątek stanowił Quick Sort, dla którego najkrótszy czas przy danych zmiennoprzecinkowych uzyskano przy 4 jednostkach wykonawczych zamiast 2.

Podobną tendencję zaobserwowano szczególnie dla Bucket Sort oraz Sample Sort. Dla obu typów danych Bucket Sort skracał czas sortowania do 16 jednostek, natomiast Sample Sort najkrótszy czas uzyskiwał przy 8 jednostkach. Zwiększenie ich liczby do 16 i 32 prowadziło już do wyraźnego wydłużenia czasu wykonania. Powtarzalność tych zależności dla dwóch typów danych sugeruje, że obserwowane zmiany wydajności mogły wynikać przede wszystkim ze sposobu realizacji równoległości badanych algorytmów, a nie wyłącznie z typu przetwarzanych wartości.

Uzyskane wyniki pokazują, że zwiększenie liczby wykorzystywanych jednostek nie prowadziło do proporcjonalnego wzrostu wydajności. Znaczenie ma również sposób zrównoleglenia konkretnego algorytmu oraz narzut związany z organizacją obliczeń równoległych. Wpływ tych czynników można ocenić również na podstawie wykorzystania zasobów oraz uzyskanego przyspieszenia i efektywności.

6.3 Analiza zużycia pamięci RAM i obciążenia CPU

Kolejnym analizowanym aspektem było wykorzystanie zasobów sprzętowych podczas działania równoległych wariantów badanych algorytmów. Uwzględniono średnie obciążenie procesora oraz średnie i maksymalne wykorzystanie pamięci RAM.

W celu bezpośredniego porównania wykorzystania zasobów przez badane algorytmy przyjęto wspólną konfigurację wykorzystującą 8 jednostek wykonawczych, stanowiącą środkową wartość spośród badanych konfiguracji.

W tabeli 6.3 przedstawiono średnie obciążenie CPU oraz wykorzystanie pamięci RAM dla zbioru `random_int` zawierającego 1 000 000 elementów.

Obciążenie procesora CPU

Wartości obciążenia CPU mogą przekraczać 100%, ponieważ pomiar uwzględnia zarówno proces główny, jak i procesy potomne wykorzystywane podczas wykonywania algorytmu równoległego. W zastosowanej metodzie pomiarowej 100% oznacza

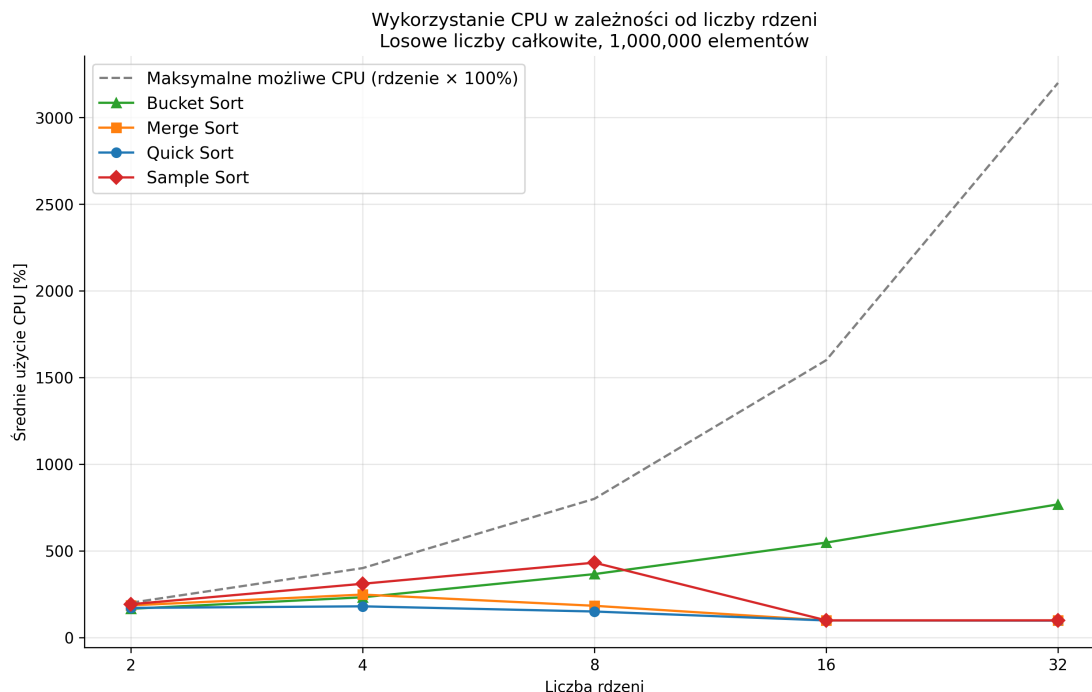
Algorytm	CPU [%]	RAM avg [MB]	RAM max [MB]
Bucket Sort	365.5027	250.3559	461.7773
Merge Sort	182.6045	163.3307	217.0391
Quick Sort	149.6238	259.0411	373.1680
Sample Sort	431.5088	265.4596	480.2227

Tabela 6.3: Średnie obciążenie CPU oraz wykorzystanie pamięci RAM dla zbioru `random_int` o rozmiarze 1 000 000 elementów i 8 jednostkach wykonawczych.

pełne obciążenie jednego procesora logicznego, natomiast wyższe wartości wskazują na równoczesne wykorzystanie kilku jednostek wykonawczych.

Przy wykorzystaniu 8 jednostek wykonawczych najwyższe średnie obciążenie CPU odnotowano dla Sample Sort i wyniosło ono około 431,51%. Drugą najwyższą wartość uzyskał Bucket Sort z wynikiem około 365,50%. Merge Sort oraz Quick Sort charakteryzowały się wyraźnie mniejszym obciążeniem procesora, osiągając odpowiednio około 182,60% i 149,62%.

Zmiany obciążenia procesora wraz ze wzrostem liczby jednostek wykonawczych przedstawiono na rysunku 6.3. Dodatkowo zaznaczono teoretyczne maksymalne obciążenie CPU, odpowiadające 100% dla każdej wykorzystywanej jednostki wykonawczej.



Rysunek 6.3: Średnie wykorzystanie CPU dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.

Źródło: opracowanie własne.

Dla Bucket Sort zwiększanie liczby jednostek wykonawczych skutkowało stop-

niowym wzrostem obciążenia procesora. Średnie wykorzystanie CPU wzrosło z około 165,35% dla 2 jednostek do 231,69% dla 4, 365,50% dla 8 oraz 547,26% dla 16 jednostek wykonawczych. Największe średnie obciążenie CPU odnotowano przy 32 jednostkach wykonawczych i wyniosło ono około 768,07%. Wyniki pokazują, że wraz ze zwiększaniem liczby jednostek wykonawczych Bucket Sort generował coraz większe łączne obciążenie procesora.

Wzrost obciążenia CPU nie prowadził jednak w każdym przypadku do dalszego skracania czasu sortowania. Dla Bucket Sort najlepszy czas uzyskano przy 16 jednostkach wykonawczych, natomiast zwiększenie ich liczby do 32 spowodowało wydłużenie średniego czasu z około 1,09 s do 1,25 s, pomimo wzrostu średniego wykorzystania procesora z około 547% do 768%. Wskazuje to, że większe wykorzystanie CPU nie zawsze wiąże się z poprawą czasu wykonania.

Odminną tendencję zaobserwowano dla Merge Sort. Wykorzystanie CPU wzrosło z około 184,96% dla 2 jednostek wykonawczych do około 247,28% dla 4 jednostek, po czym przy 8 jednostkach spadło do około 182,60%. Dla 16 oraz 32 jednostek wykonawczych średnie obciążenie procesora osiągało już tylko około 98%. Spadkowi wykorzystania CPU towarzyszyło jednocześnie pogorszenie czasu sortowania, co odpowiada wynikom przedstawionym w sekcji 6.2.

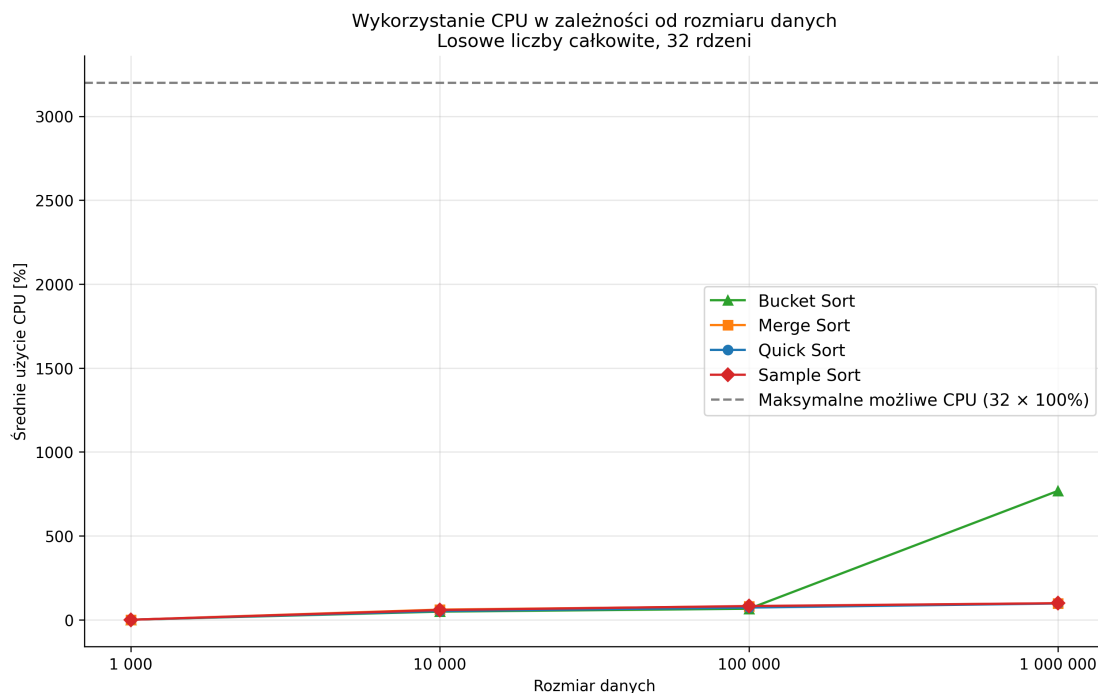
Podobną zależność zaobserwowano dla Quick Sort. Wyższe obciążenie procesora występowało przy mniejszej liczbie jednostek wykonawczych. Dla 2 jednostek wynosiło ono około 170,36%, natomiast dla 4 osiągnęło około 179,53%. Przy 8 jednostkach wartość zmniejszyła się do około 149,62%, a dla 16 i 32 wynosiła już około 98%. Wraz ze zwiększaniem liczby jednostek wykonawczych nie obserwowano odpowiadającego mu wzrostu obciążenia procesora.

Wyraźne zmiany wystąpiły również dla Sample Sort. Średnie obciążenie procesora wzrosło z około 191,57% dla 2 jednostek wykonawczych do 309,72% dla 4, a najwyższy poziom około 431,51% odnotowano przy 8 jednostkach. Dalsze zwiększenie ich liczby do 16 i 32 skutkowało zmniejszeniem wykorzystania CPU do około 99%.

Porównanie wyników wskazuje zatem na wyraźne różnice w sposobie wykorzystania dostępnej równoległości przez poszczególne algorytmy. Bucket Sort jako jedyny zwiększał średnie obciążenie CPU w całym badanym zakresie od 2 do 32 jednostek wykonawczych.

Kolejnym czynnikiem wpływającym na wykorzystanie procesora był rozmiar przetwarzanego zbioru. Zależność średniego obciążenia CPU od liczby sortowanych elementów dla 32 jednostek wykonawczych przedstawiono na rysunku 6.4.

Dla najmniejszych zbiorów danych w części pomiarów odnotowano bardzo niskie, a w niektórych przypadkach również zerowe wartości średniego wykorzystania CPU. Wyniki te należy interpretować z uwzględnieniem przyjętego sposobu próbkowania obciążenia procesora. Ze względu na krótki czas wykonania sortowania dla



Rysunek 6.4: Średnie wykorzystanie CPU w zależności od rozmiaru zbioru `random_int` dla 32 jednostek wykonawczych.

Źródło: opracowanie własne.

najmniejszych zbiorów pomiar mógł zakończyć się przed zarejestrowaniem wystarczającej liczby próbek. Wartość równa 0% nie oznacza zatem braku wykorzystania procesora, lecz wynika z ograniczenia zastosowanej metody pomiarowej. Dla większych zbiorów danych rejestrowane wartości wykorzystania procesora były wyższe. Szczególnie widoczne było to dla Bucket Sort, dla którego przy 32 jednostkach wykonawczych średnie wykorzystanie CPU wzrosło z około 65,63% dla 100 000 elementów do około 768,07% dla 1 000 000 elementów.

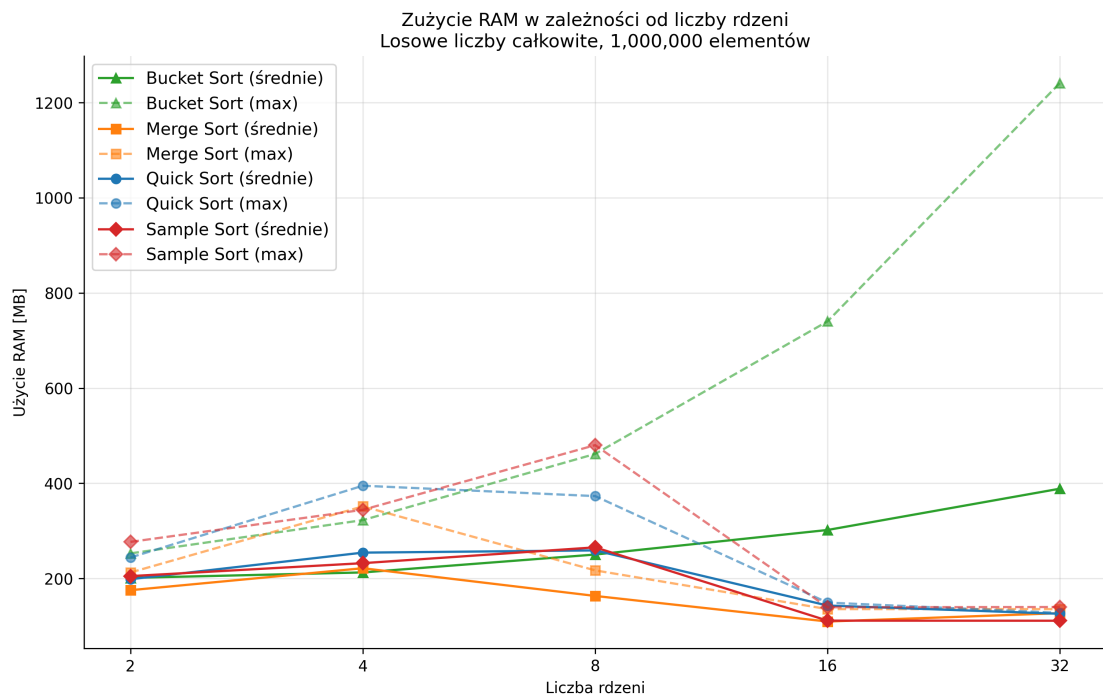
Dla Merge Sort, Quick Sort oraz Sample Sort przy 32 jednostkach wykonawczych i 1 000 000 elementów średnie wykorzystanie procesora wynosiło natomiast około 97–99%. Oznacza to, że w tych konfiguracjach zwiększona liczba jednostek wykonawczych nie przełożyła się na wyższe obciążenie CPU. Zaobserwowane różnice pomiędzy algorytmami wskazują na znaczenie sposobu organizacji obliczeń równoległych.

Zbliżony przebieg zaobserwowano również dla zbioru `random_float`. Bucket Sort zwiększał wykorzystanie CPU wraz ze wzrostem liczby jednostek wykonawczych, natomiast dla Sample Sort najwyższe obciążenie procesora wystąpiło przy 8 jednostkach. Wskazuje to, że zaobserwowane zależności nie były charakterystyczne wyłącznie dla danych typu całkowitego.

Wykorzystanie pamięci RAM

Dla przyjętej wcześniej wspólnej konfiguracji 8 jednostek wykonawczych przeanalizowano również wykorzystanie pamięci RAM przez poszczególne algorytmy. Największe średnie zużycie pamięci odnotowano dla Sample Sort i wyniosło ono około 265,46 MB. Zbliżony wynik uzyskał Quick Sort z wartością około 259,04 MB, natomiast dla Bucket Sort było to około 250,36 MB. Najniższe średnie wykorzystanie pamięci zarejestrowano dla Merge Sort i wyniosło około 163,33 MB. Pod względem maksymalnego zużycia pamięci najwyższe wartości odnotowano dla Sample Sort oraz Bucket Sort, odpowiednio około 480,22 MB i 461,78 MB.

Wpływ zwiększania liczby jednostek wykonawczych na wykorzystanie pamięci RAM przedstawiono na rysunku 6.5.



Rysunek 6.5: Średnie i maksymalne wykorzystanie pamięci RAM dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.

Źródło: opracowanie własne.

Najbardziej wyraźny wzrost wykorzystania pamięci wraz z liczbą jednostek wykonawczych wystąpił dla Bucket Sort. Dla zbioru zawierającego 1 000 000 elementów średnie wykorzystanie pamięci wzrosło z około 201,28 MB dla 2 jednostek wykonawczych do 212,73 MB dla 4 oraz 250,36 MB dla 8. Dla 16 jednostek wyniosło około 301,95 MB, a przy 32 wzrosło do około 388,71 MB.

Jeszcze wyraźniejszy wzrost zaobserwowano w przypadku maksymalnego zużycia pamięci przez Bucket Sort. Wartość ta wzrosła z około 252,76 MB dla 2 jednostek wykonawczych do 322,67 MB dla 4, 461,78 MB dla 8 oraz 740,36 MB dla

16 jednostek. Przy 32 jednostkach maksymalne wykorzystanie pamięci wzrosło do około 1241,26 MB.

Dalsze zwiększenie liczby jednostek wykonawczych z 16 do 32 nie przełożyło się na krótszy czas sortowania. Wyniki wskazują, że zwiększenie poziomu równoległości może wiązać się ze znacznym wzrostem wykorzystania pamięci, bez jednoczesnego skrócenia czasu wykonania.

W przypadku pozostałych algorytmów zależność pomiędzy liczbą jednostek wykonawczych a wykorzystaniem pamięci nie była tak jednoznaczna. Dla części badanych konfiguracji najwyższe wartości uzyskiwano przy pośredniej liczbie jednostek wykonawczych, natomiast przy 16 lub 32 jednostkach były one niższe. Było to szczególnie widoczne dla Quick Sort oraz Sample Sort.

Podobną tendencję zaobserwowano dla zbioru `random_float` zawierającego 1 000 000 elementów, gdzie wykorzystanie pamięci RAM przez Bucket Sort zwiększało się wraz z liczbą jednostek wykonawczych. Przy 32 jednostkach wykonawczych maksymalne wykorzystanie pamięci osiągnęło około 1245,42 MB, co stanowiło wynik zbliżony do 1241,26 MB uzyskanych dla danych całkowitych.

Podobieństwo wyników dla obu typów danych wskazuje, że wzrost zapotrzebowania na pamięć w przypadku Bucket Sort był związany głównie ze zwiększaniem liczby jednostek wykonawczych, natomiast typ sortowanych wartości miał w tym przypadku mniejsze znaczenie.

Uzyskane wyniki pokazują, że wykorzystanie zasobów różniło się w zależności od badanego algorytmu. Zwiększenie liczby jednostek wykonawczych nie w każdym przypadku powodowało wzrost obciążenia CPU, a dla niektórych konfiguracji jego wartość ulegała zmniejszeniu. Inaczej zachowywał się Bucket Sort. W jego przypadku wzrost liczby jednostek skutkował większym wykorzystaniem zarówno procesora, jak i pamięci RAM.

Na wykorzystanie zasobów wpływał również rozmiar danych wejściowych. Wyższe obciążenie CPU obserwowano przede wszystkim dla największego badanego zbioru, podczas gdy dla mniejszych zbiorów możliwości wykorzystania wielu jednostek wykonawczych pozostawały ograniczone.

Wyniki pokazują zatem, że wpływ zwiększania poziomu równoległości na wykorzystanie zasobów był zależny od badanego algorytmu. Większa liczba jednostek wykonawczych nie gwarantowała ich pełnego wykorzystania, a w niektórych przypadkach wiązała się ze wzrostem zapotrzebowania na pamięć RAM.

6.4 Skalowalność algorytmów

Kolejnym elementem analizy była ocena skalowalności równoległych wariantów badanych algorytmów. W tym celu wykorzystano wartości przyspieszenia oraz efektywności uzyskane dla różnych liczb jednostek wykonawczych i rozmiarów danych

wejściowych.

Analizę przeprowadzono w dwóch ujęciach. W pierwszym przypadku analizowano zmiany przyspieszenia i efektywności wynikające ze wzrostu liczby jednostek wykonawczych przy niezmiennym rozmiarze danych. W drugim natomiast oceniono, jak zwiększanie rozmiaru danych wejściowych wpływa na wyniki z zachowaniem tej samej liczby jednostek wykonawczych.

Szczegółową analizę przeprowadzono dla zbioru `random_int`, natomiast wyniki dla zbioru `random_float` posłużyły do oceny, czy podobne zależności można zaobserwować również dla danych zmiennoprzecinkowych.

Skalowalność względem liczby jednostek wykonawczych

W pierwszej kolejności przeanalizowano skalowalność algorytmów przy rosnącej liczbie jednostek wykonawczych dla zbioru o stałym rozmiarze 1 000 000 elementów. Uwzględniono konfiguracje obejmujące 2, 4, 8, 16 i 32 jednostki wykonawcze.

Analiza przyspieszenia pozwala określić, jak zwiększanie liczby jednostek wykonawczych wpływało na wydajność poszczególnych algorytmów. Wyniki zestawiono w tabeli 6.4.

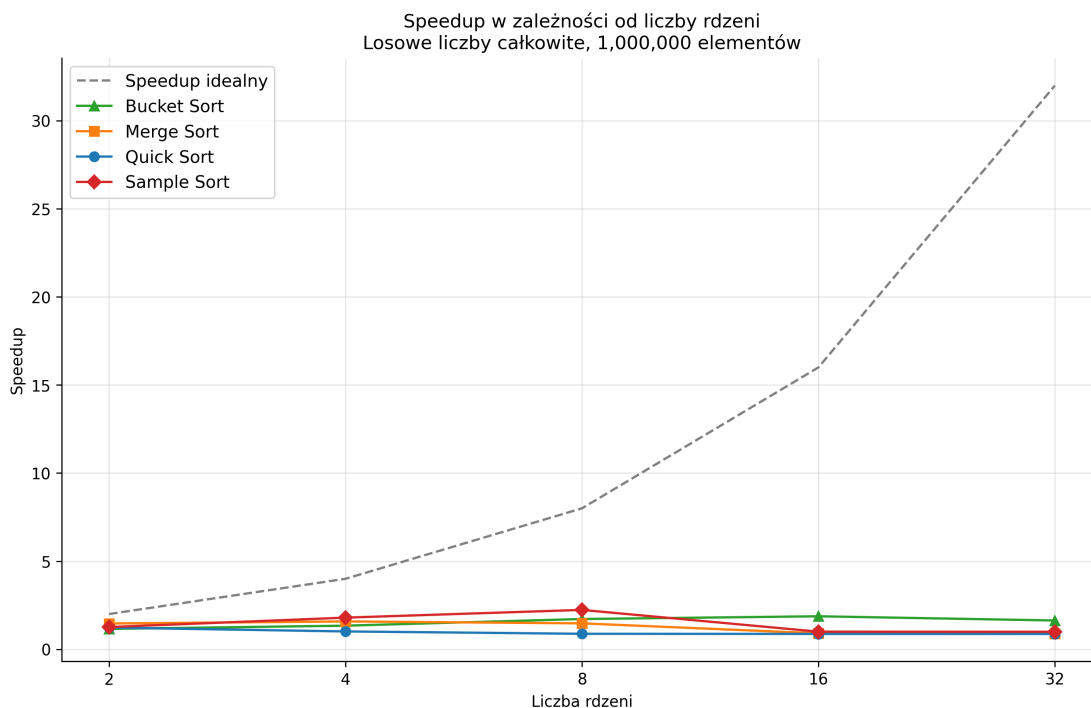
Rdzenie	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
2	1.2284	1.4616	1.1574	1.2586
4	1.0122	1.5795	1.3384	1.7951
8	0.8779	1.4712	1.7171	2.2336
16	0.8703	0.9030	1.8718	0.9985
32	0.8688	0.8865	1.6325	0.9919

Tabela 6.4: Przyspieszenie badanych algorytmów w zależności od liczby jednostek wykonawczych.

Przebieg zmian przyspieszenia wraz ze wzrostem jednostek wykonawczych przedstawiono również na rysunku 6.6. Linia przerywaną oznaczono przyspieszenie idealne, zakładające wzrost wartości przyspieszenia proporcjonalnie do liczby jednostek wykonawczych.

Uzyskane wyniki wskazują, że zwiększanie liczby jednostek wykonawczych nie prowadziło do proporcjonalnego wzrostu przyspieszenia. Dla każdego z badanych algorytmów najwyższa wartość przyspieszenia wystąpiła przy innej liczbie jednostek wykonawczych.

W przypadku Quick Sort najwyższe przyspieszenie, wynoszące około 1,23, uzyskano dla 2 jednostek wykonawczych. Dla 4 jednostek wartość przyspieszenia wyniosła około 1,01, natomiast przy większej liczbie jednostek spadła poniżej 1. Dla 8, 16 i 32 jednostek osiągnęła odpowiednio około 0,88, 0,87 i 0,87, co oznacza, że



Rysunek 6.6: Przyspieszenie badanych algorytmów dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.

Źródło: opracowanie własne.

w tych konfiguracjach wariant równoległy wykonywał sortowanie wolniej od wariantu sekwencyjnego.

Merge Sort osiągnął najwyższe przyspieszenie przy 4 jednostkach wykonawczych, dla których wyniosło ono około 1,58. Dla 2 oraz 8 jednostek uzyskano odpowiednio około 1,46 i 1,47. Przy dalszym zwiększaniu liczby jednostek wykonawczych zaobserwowano spadek wartości przyspieszenia do poziomu około 0,90 dla 16 jednostek oraz 0,89 dla 32 jednostek. Uzyskane rezultaty wskazują, że przekroczenie liczby 8 jednostek wykonawczych nie przyczyniało się już do poprawy wydajności analizowanej implementacji.

Odmienne przebieg zaobserwowano dla Bucket Sort. Wartość przyspieszenia wzrastała od około 1,16 dla 2 jednostek wykonawczych do 1,34 dla 4 oraz 1,72 dla 8 jednostek. Najwyższą wartość, wynoszącą około 1,87, uzyskano dla 16 jednostek wykonawczych. Zwiększenie liczby jednostek wykonawczych do 32 doprowadziło do spadku wartości przyspieszenia do poziomu około 1,63. Bucket Sort jako jedyny z badanych algorytmów zachował przyspieszenie większe od 1 dla wszystkich analizowanych konfiguracji.

Spośród wszystkich analizowanych algorytmów najwyższą wartość przyspieszenia osiągnął Sample Sort. Wartość ta zwiększyła się z około 1,26 dla 2 jednostek wykonawczych do 1,80 dla 4 oraz osiągnęła 2,23 przy 8 jednostkach. Dalsze zwiększenie liczby jednostek spowodowało jednak wyraźny spadek przyspieszenia. Dla 16 jednostek wykonawczych wartość ta wynosiła około 1,00, natomiast dla 32

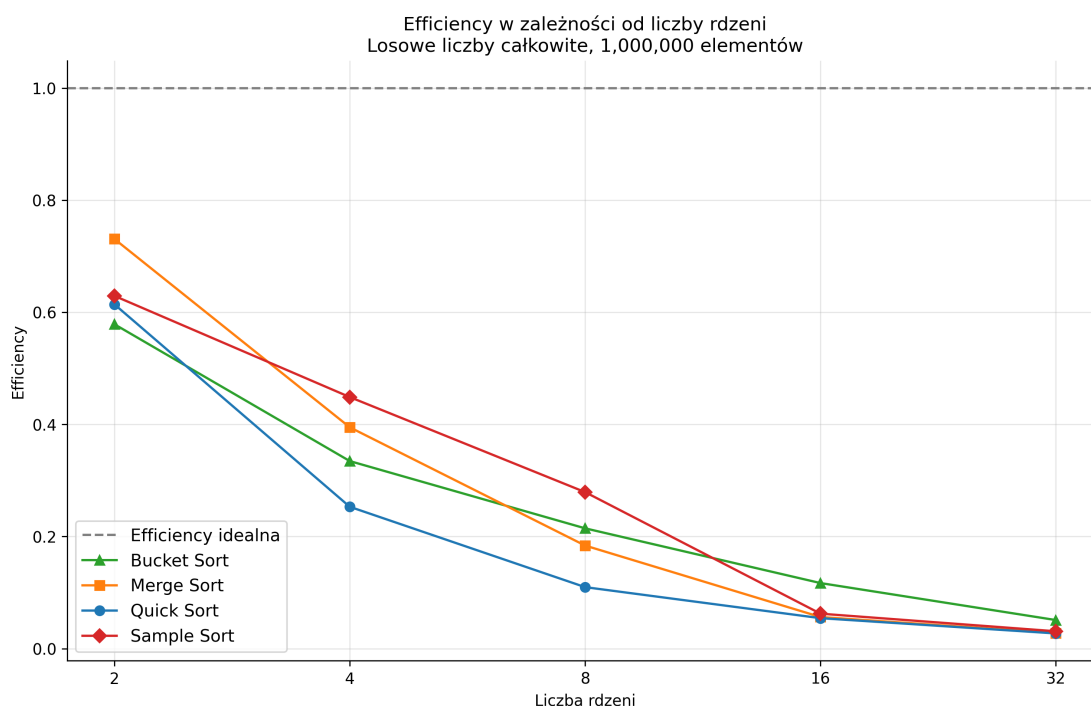
jednostek około 0,99. Najkorzystniejszą konfiguracją Sample Sort spośród przebadanych okazało się zatem zastosowanie 8 jednostek wykonawczych.

Kolejną miarą wykorzystaną do oceny skalowalności była efektywność, określająca przyspieszenie przypadające na jedną wykorzystywaną jednostkę wykonawczą. Porównanie uzyskanych wartości dla badanych algorytmów zestawiono w tabeli 6.5.

Rdzenie	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
2	0.6142	0.7308	0.5787	0.6293
4	0.2531	0.3949	0.3346	0.4488
8	0.1097	0.1839	0.2146	0.2792
16	0.0544	0.0564	0.1170	0.0624
32	0.0272	0.0277	0.0510	0.0310

Tabela 6.5: Efektywność badanych algorytmów w zależności od liczby jednostek wykonawczych.

Zmiany efektywności wraz ze zwiększaniem liczby jednostek wykonawczych przedstawiono na rysunku 6.7. Wartość równa 1 oznacza efektywność idealną, w której zwiększenie liczby jednostek wykonawczych skutkowałoby proporcjonalnym przyrostem przyspieszenia.



Rysunek 6.7: Efektywność badanych algorytmów dla zbioru `random_int` o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.

Źródło: opracowanie własne.

Dla wszystkich badanych algorytmów wraz ze zwiększaniem liczby jednostek wykonawczych obserwowano spadek efektywności. Najwyższe wartości występowały dla konfiguracji wykorzystującej 2 jednostki. Wynosiły one około 0,73 dla Merge Sort, 0,63 dla Sample Sort, 0,61 dla Quick Sort oraz 0,58 dla Bucket Sort.

Przy 4 jednostkach wykonawczych najwyższą efektywność osiągnął Sample Sort, dla którego wyniosła ona około 0,45. Merge Sort uzyskał około 0,39, Bucket Sort około 0,33, natomiast Quick Sort około 0,25. Przy 8 jednostkach wartości spadły odpowiednio do około 0,28 dla Sample Sort, 0,21 dla Bucket Sort, 0,18 dla Merge Sort oraz 0,11 dla Quick Sort.

Dalsze zwiększanie liczby jednostek wykonawczych skutkowało wyraźnym obniżeniem efektywności. Przy 32 jednostkach najwyższą wartość uzyskał Bucket Sort, jednak wynosiła ona jedynie około 0,05. W przypadku Sample Sort, Merge Sort oraz Quick Sort efektywność utrzymywała się na poziomie około 0,03.

Obserwowany spadek efektywności wraz ze zwiększaniem liczby jednostek wykonawczych wskazuje, że dodatkowe zasoby obliczeniowe nie przekładały się na proporcjonalny przyrost przyspieszenia. Największe różnice były widoczne dla konfiguracji wykorzystujących 16 i 32 jednostki, dla których efektywność wszystkich badanych algorytmów pozostawała na niskim poziomie.

W porównaniu z pozostałymi algorytmami Bucket Sort wyróżniał się najbardziej równomiernym wzrostem przyspieszenia wraz ze zwiększaniem liczby jednostek wykonawczych. Sample Sort uzyskał natomiast najwyższą wartość przyspieszenia spośród wszystkich badanych konfiguracji, wynoszącą około 2,23 przy 8 jednostkach wykonawczych.

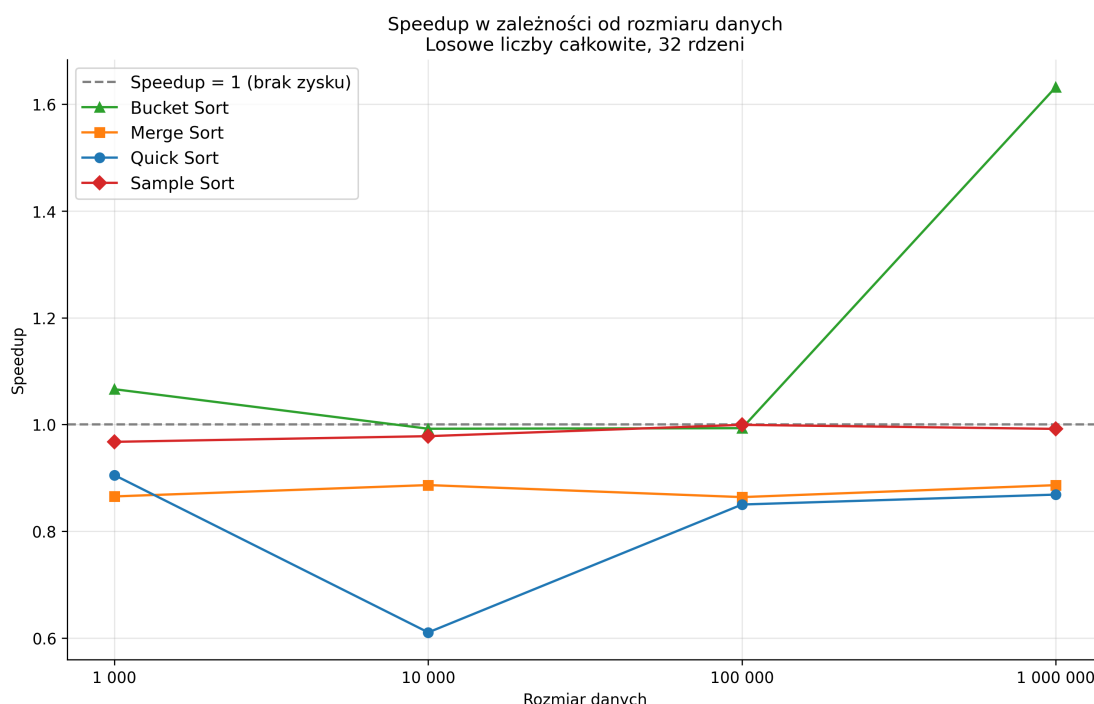
Uzyskane wartości przyspieszenia i efektywności świadczą o ograniczonej skalowalności badanych implementacji. Żaden z algorytmów nie zbliżył się do skalowania idealnego, a wraz ze wzrostem liczby jednostek wykonawczych różnica pomiędzy uzyskanym a teoretycznym przyspieszeniem ulegała zwiększeniu.

Wpływ rozmiaru danych na skalowalność

Drugim analizowanym czynnikiem był wpływ rozmiaru danych wejściowych na skalowalność badanych algorytmów. W tym celu porównano wartości przyspieszenia uzyskane przy niezmienniej liczbie 32 jednostek wykonawczych dla zbiorów liczących 1 000, 10 000, 100 000 oraz 1 000 000 elementów.

Zmiany wartości przyspieszenia w zależności od rozmiaru zbioru `random_int` przedstawiono na rysunku 6.8. Linia przerywaną oznaczono poziom przyspieszenia równy 1, przy którym czas wykonania wariantu równoległego i sekwencyjnego jest taki sam.

Uzyskane rezultaty pokazują, że wpływ zwiększania rozmiaru danych na przyspieszenie różnił się w zależności od analizowanego algorytmu. Najbardziej zauwa-



Rysunek 6.8: Przyspieszenie badanych algorytmów w zależności od rozmiaru zbioru `random_int` dla 32 jednostek wykonawczych.

Źródło: opracowanie własne.

żalną zmianę odnotowano dla Bucket Sort. Dla zbioru zawierającego 1 000 elementów przyspieszenie wynosiło około 1,07, natomiast dla 10 000 i 100 000 elementów pozostawało na poziomie zbliżonym do 1. Dla największego badanego zbioru, zawierającego 1 000 000 elementów, wartość ta wzrosła do około 1,63. Oznacza to, że dla Bucket Sort konfiguracja obejmująca 32 jednostki wykonawcze stawiała się korzystniejsza wraz ze wzrostem rozmiaru danych.

Inną zależność zaobserwowano dla Merge Sort. W całym analizowanym zakresie wartość przyspieszenia utrzymywała się poniżej 1 i wynosiła około 0,86–0,89. Zwiększenie rozmiaru danych do 1 000 000 elementów nie przełożyło się zatem na uzyskanie przewagi wariantu równoległego nad wariantem sekwencyjnym przy 32 jednostkach wykonawczych.

W przypadku Quick Sort przyspieszenie również pozostawało poniżej 1 dla wszystkich analizowanych rozmiarów danych. Dla 1 000 elementów wynosiło około 0,90, natomiast dla 10 000 elementów zmniejszyło się do około 0,61. Dla większych zbiorów jego wartość ponownie się zwiększyła, osiągając około 0,85 dla 100 000 oraz 0,87 dla 1 000 000 elementów. Mimo odnotowanego wzrostu wariant równoległy nie osiągnął przy 32 jednostkach wykonawczych przyspieszenia przekraczającego wartość 1.

Sample Sort charakteryzował się wartościami przyspieszenia zbliżonymi do 1. Dla 1 000 elementów wynosiło ono około 0,97, a dla 10 000 elementów około 0,98. Przy 100 000 elementów wartość osiągnęła poziom bliski 1, natomiast dla 1 000 000

elementów wyniosła około 0,99. Wzrost rozmiaru danych nie przyniósł zatem w tej konfiguracji wyraźnego zwiększenia przyspieszenia.

Porównanie uzyskanych rezultatów wskazuje, że zwiększanie rozmiaru danych wejściowych w zróżnicowanym stopniu oddziaływało na skalowalność badanych algorytmów. Największe korzyści przy zastosowaniu 32 jednostek wykonawczych odnotowano dla Bucket Sort, który dla największego zbioru osiągnął przyspieszenie przekraczające 1. W przypadku pozostałych algorytmów nawet zwiększenie rozmiaru danych do 1 000 000 elementów nie umożliwiło uzyskania przewagi wariantu równoległego nad sekwencyjnym.

Zbliżone zależności zaobserwowano również dla zbioru `random_float`. Wyniki uzyskane dla danych zmiennoprzecinkowych potwierdziły, że wpływ rozmiaru danych na skalowalność zależał od analizowanego algorytmu, a zwiększenie liczby przetwarzanych elementów nie gwarantowało poprawy przyspieszenia we wszystkich implementacjach.

Uzyskane wyniki wskazują na zróżnicowaną skalowalność badanych implementacji. Wraz ze zwiększaniem liczby jednostek wykonawczych we wszystkich algorytmach odnotowywano spadek efektywności, jednak tempo tego spadku oraz zmiany uzyskiwanego przyspieszenia różniły się pomiędzy poszczególnymi algorytmami.

Najkorzystniejszą skalowalność w zakresie większej liczby jednostek wykonawczych wykazywał Bucket Sort, który jako jedyny przy 16 oraz 32 jednostkach zachował przyspieszenie większe od 1. Pozostałe algorytmy osiągały najwyższe wartości przyspieszenia przy mniejszej liczbie jednostek, po czym dalsze zwiększanie poziomu równoległości prowadziło do spadku uzyskiwanych wartości.

Analiza wpływu rozmiaru danych pokazała ponadto, że zwiększanie liczby przetwarzanych elementów nie zapewniało jednakowej poprawy skalowalności wszystkich algorytmów. Najbardziej wyraźna korzyść dla największego analizowanego zbioru wystąpiła w przypadku Bucket Sort, natomiast dla pozostałych implementacji zwiększenie rozmiaru danych nie okazało się wystarczające do efektywnego wykorzystania konfiguracji obejmującej 32 jednostki wykonawcze.

6.5 Wpływ typu i charakterystyki danych wejściowych

Kolejnym etapem analizy było określenie wpływu typu i charakterystyki danych wejściowych na czas działania badanych algorytmów równoległych. Porównano wyniki otrzymane dla liczb całkowitych i zmiennoprzecinkowych oraz zbiorów losowych, zawierających duplikaty i częściowo uporządkowanych.

W celu bezpośredniego porównania zachowania algorytmów przy różnych charakterystykach danych przyjęto wspólną konfigurację obejmującą 1 000 000 elementów oraz 8 jednostek wykonawczych.

W tabeli 6.6 przedstawiono średnie czasy wykonania badanych algorytmów dla analizowanych rodzajów danych. Umożliwiło to zbadanie oddziaływania typu danych wejściowych przy utrzymaniu niezmiennego rozmiaru zbioru oraz stopnia równoległości.

Zbiór danych	Quick Sort	Merge Sort	Bucket Sort	Sample Sort
20% posortowanych (float)	2.0954	2.2794	1.1927	2.2278
20% posortowanych (int)	2.1256	2.2961	1.1664	2.2737
40% posortowanych (float)	2.1120	2.2273	1.1601	2.2110
40% posortowanych (int)	2.4171	2.2491	1.1625	2.2648
60% posortowanych (float)	2.2085	2.1864	1.1590	2.2188
60% posortowanych (int)	2.4828	2.1818	1.1563	2.2424
80% posortowanych (float)	1.6647	2.0946	1.1188	2.2046
80% posortowanych (int)	1.6474	2.1279	1.1192	2.1541
Duplikaty (float)	2.7048	2.3373	1.1857	2.1799
Duplikaty (int)	2.3277	2.3555	1.1666	2.2038
Losowe liczby (float)	2.4445	2.3422	1.1991	2.1750
Losowe liczby (int)	2.5613	2.3212	1.1885	2.2156

Tabela 6.6: Średni czas wykonania badanych algorytmów dla różnych charakterystyk danych wejściowych przy 8 jednostkach wykonawczych.

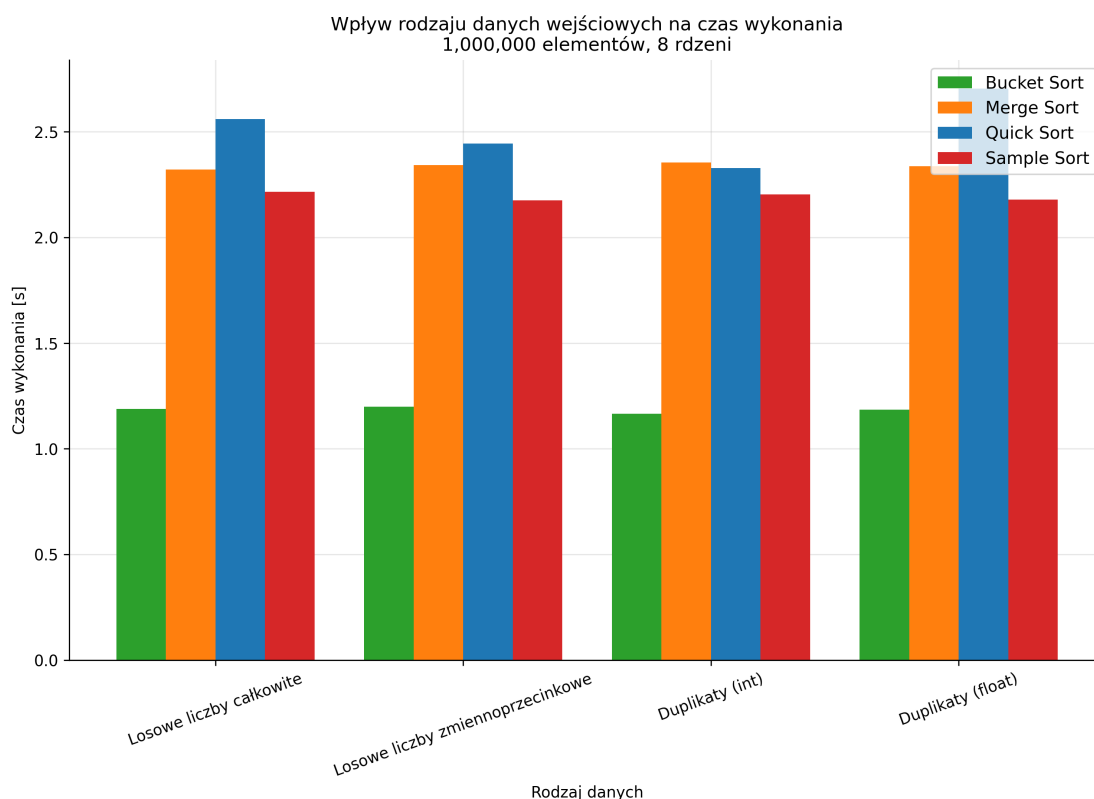
Analizę przeprowadzono w dwóch etapach. Na początku porównano wyniki dla danych losowych i zbiorów zawierających duplikaty, z uwzględnieniem danych całkowitych i zmiennoprzecinkowych. W dalszej kolejności przeanalizowano wpływ poziomu częściowego uporządkowania danych na czas wykonania badanych algorytmów.

Wpływ typu danych i występowania duplikatów

Porównanie danych losowych oraz zbiorów zawierających duplikaty pozwoliło ocenić wpływ typu wartości i występowania powtarzających się elementów na czas wykonania badanych algorytmów.

Uzyskane rezultaty wskazują, że wpływ typu danych oraz obecności duplikatów był zróżnicowany w zależności od analizowanego algorytmu. Dla Merge Sort, Bucket Sort i Sample Sort różnice pomiędzy analizowanymi rodzajami danych były stosunkowo niewielkie. Większe zmiany zaobserwowano natomiast w przypadku Quick Sort.

Zestawienie czasów wykonania dla poszczególnych rodzajów danych przedstawiono również na rysunku 6.9.



Rysunek 6.9: Wpływ typu danych i występowania duplikatów na czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych.

Źródło: opracowanie własne.

W przypadku Merge Sort czasy wykonania dla wszystkich czterech badanych zbiorów były bardzo zbliżone i zawierały się w przedziale około 2,32–2,36 s. Podobną sytuację zaobserwowano dla Bucket Sort, dla którego średni czas wykonania wynosił około 1,17–1,20 s. Niewielkie różnice pojawiły się również dla Sample Sort, którego wyniki mieściły się w zakresie około 2,18–2,22 s. Oznacza to, że w przypadku tych algorytmów zmiana typu wartości oraz obecność duplikatów wywierały ograniczony wpływ na czas wykonania w analizowanej konfiguracji.

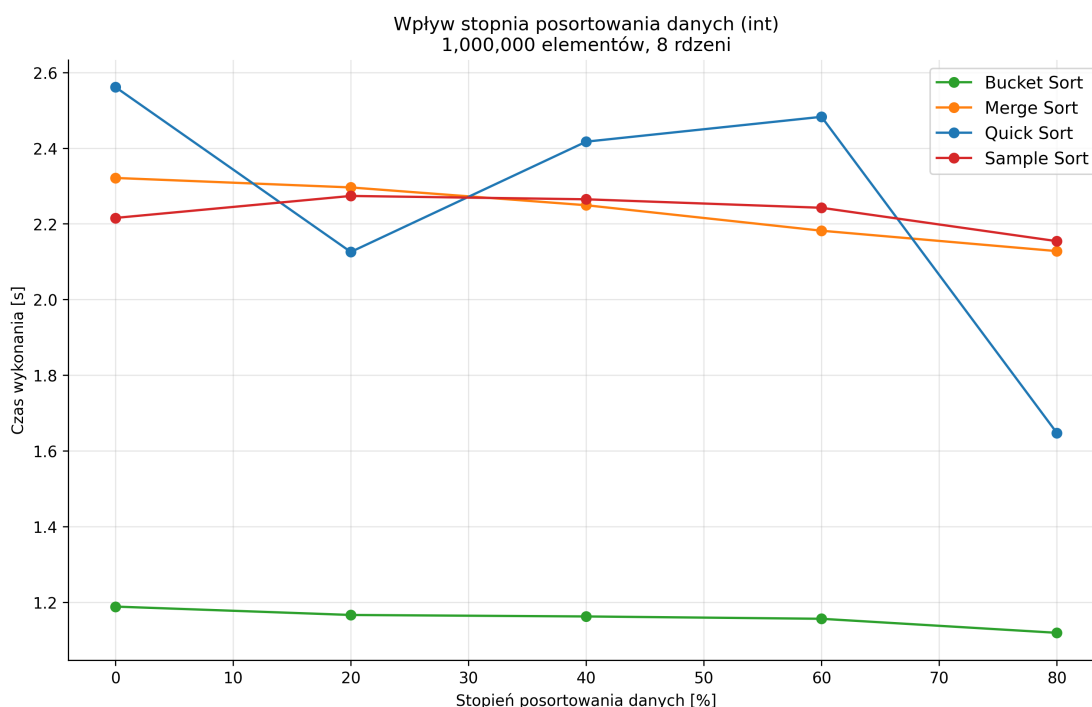
Większe różnice wystąpiły dla Quick Sort. Dla losowych liczb całkowitych średni czas wykonania wyniósł około 2,56 s, natomiast przy losowych liczbach zmiennoprzecinkowych było to już około 2,44 s. W przypadku zbioru zawierającego duplikaty wartości całkowite czas obniżył się do około 2,33 s, natomiast dla duplikatów zmiennoprzecinkowych zwiększył do około 2,70 s. Uzyskane wyniki pokazują, że Quick Sort charakteryzował się większą zmiennością czasu wykonania pomiędzy analizowanymi rodzajami danych niż pozostałe badane algorytmy.

Porównanie uzyskanych wyników nie wskazuje na jednoznaczną przewagę danych całkowitych bądź zmiennoprzecinkowych dla wszystkich analizowanych algorytmów. Podobnie obecność duplikatów nie wywoływała jednakowej zmiany czasu wykonania w każdym przypadku.

Wpływ stopnia częściowego uporządkowania danych

Kolejnym analizowanym czynnikiem był stopień częściowego uporządkowania danych wejściowych. Posłużyły do tego zbiory, w których uporządkowane było odpowiednio 20%, 40%, 60% oraz 80% elementów. Analizę wykonano dla danych całkowitych i zmiennoprzecinkowych, co umożliwiło ustalenie, czy zwiększanie stopnia uporządkowania zbioru oddziaływało na czas wykonania badanych algorytmów.

Zależność pomiędzy stopniem częściowego uporządkowania danych a czasem wykonania poszczególnych algorytmów przedstawiono na rysunku 6.10.



Rysunek 6.10: Wpływ stopnia częściowego uporządkowania danych całkowitych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych.

Źródło: opracowanie własne.

Widoczny wpływ stopnia uporządkowania danych występował dla algorytmu Quick Sort. Dla zbioru zawierającego 20% uporządkowanych elementów średni czas wykonania wyniósł około 2,13 s. Przy 40% oraz 60% uporządkowania czas wzrósł odpowiednio do około 2,42 s i 2,48 s. Inny rezultat odnotowano dla zbioru uporządkowanego w 80%, dla którego średni czas wykonania spadł do około 1,65 s. Wyniki nie wykazały regularnej zależności pomiędzy wzrostem uporządkowania danych a czasem działania Quick Sort. Najlepsze rezultaty osiągnięto jednak dla najwyższego stopnia uporządkowania, który dał najkrótszy czas spośród wszystkich testowanych wariantów.

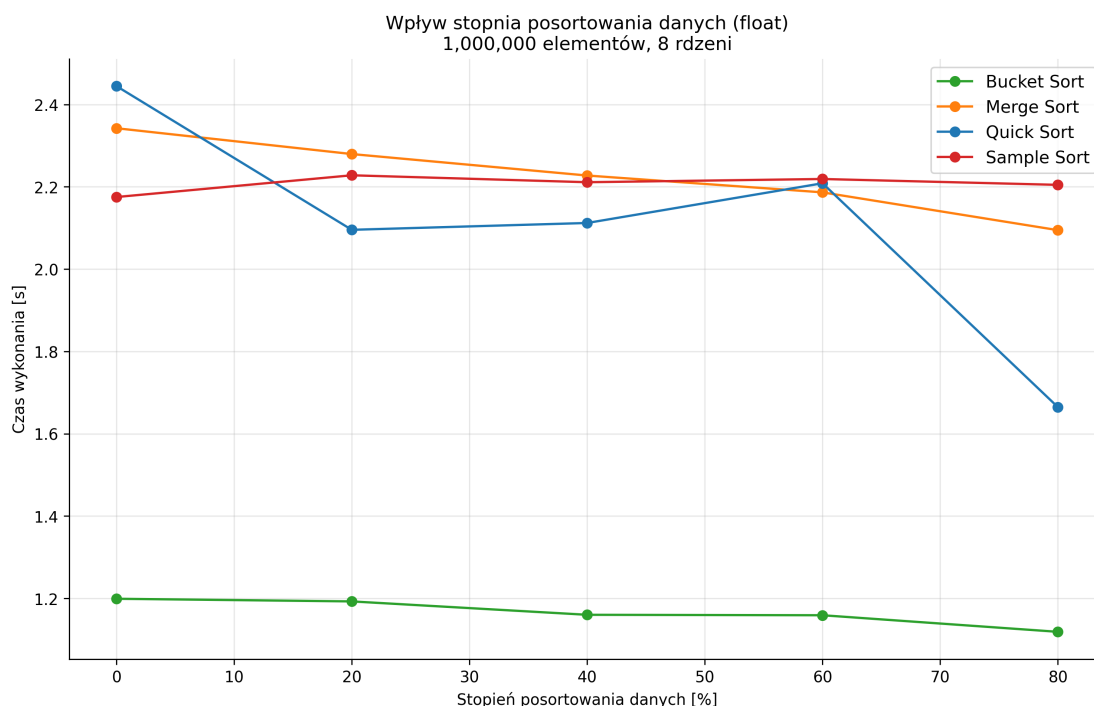
W przypadku Merge Sort wzrost stopnia uporządkowania danych skutkował nieznacznym skróceniem czasu wykonania. Średni czas spadł z około 2,30 s dla

20% uporządkowanych elementów do około 2,25 s dla 40%, 2,18 s dla 60% oraz 2,13 s dla 80%. Zaobserwowane zmiany były jednak mniej wyraźne niż dla Quick Sort.

Bucket Sort z kolei charakteryzował się dużą stabilnością uzyskiwanych wyników. Zbiory uporządkowane w 20%, 40% i 60% posiadały średni czas wykonania na poziomie około 1,16 s. Przy 80% uporządkowania zmniejszył się jednak do około 1,12 s. Oznacza to, że w badanym zakresie stopień częściowego uporządkowania danych nie miał dużego wpływu na czas wykonania tego algorytmu.

Podobnie małe różnice pomiędzy wynikami wystąpiły dla Sample Sort. Średni czas wykonania wynosił około 2,27 s dla 20% uporządkowanych elementów, 2,26 s dla 40%, 2,24 s dla 60% oraz 2,15 s dla 80%. Wraz ze wzrostem stopnia uporządkowania czas wykonania nieznacznie się skracał, jednak różnice pomiędzy poszczególnymi zbiorami były niewielkie.

Analogiczne wyniki dla danych zmiennoprzecinkowych przedstawiono na rysunku 6.11.



Rysunek 6.11: Wpływ stopnia częściowego uporządkowania danych zmiennoprzecinkowych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych.

Źródło: opracowanie własne.

W przypadku danych zmiennoprzecinkowych wpływ stopnia uporządkowania również różnił się pomiędzy poszczególnymi algorytmami. W przypadku Quick Sort czasy dla 20%, 40% i 60% uporządkowania wynosiły odpowiednio około 2,10 s, 2,11 s oraz 2,21 s. Przy 80% uporządkowania czas wyniósł z kolei około 1,66 s. Podobnie jak dla danych całkowitych najkrótszy czas uzyskano więc dla najwyższego

analizowanego stopnia uporządkowania.

Dla Merge Sort widoczna była stopniowa poprawa czasu wykonania. Średni czas zmniejszył się z około 2,28 s dla 20% uporządkowanych elementów do około 2,23 s dla 40%, 2,19 s dla 60% oraz 2,09 s dla 80%. Podobny przebieg zaobserwowano również dla danych całkowitych.

W przypadku Bucket Sort różnice pomiędzy poszczególnymi stopniami uporządkowania danych ponownie były niewielkie. Średnie czasy wykonania wyniosły około 1,19 s, 1,16 s, 1,16 s oraz 1,12 s odpowiednio dla 20%, 40%, 60% i 80% uporządkowania. Oznacza to, że również dla danych zmiennoprzecinkowych oddziaływanie tego czynnika na czas działania Bucket Sort pozostawało stosunkowo niewielkie.

Dla Sample Sort uzyskano również niewielkie różnice pomiędzy wynikami. Średni czas wykonania wynosił około 2,23 s dla 20% uporządkowania, 2,21 s dla 40%, 2,22 s dla 60% oraz 2,20 s dla 80%. W odróżnieniu od Quick Sort i Merge Sort wzrost stopnia uporządkowania nie prowadził zatem do wyraźnych zmian czasu wykonania tego algorytmu.

Zestawienie wyników dla obu typów danych pokazuje różne reakcje badanych algorytmów na wzrost stopnia częściowego uporządkowania. Najbardziej widoczne różnice pomiędzy kolejnymi poziomami uporządkowania odnotowano dla Quick Sort. Zarówno dla danych całkowitych, jak i zmiennoprzecinkowych najkrótszy czas osiągnął zbiór uporządkowany w 80%.

W przypadku Merge Sort wraz ze wzrostem stopnia uporządkowania czas wykonania nieznacznie się skracał. Bucket Sort i Sample Sort wykazywały natomiast mniejsze różnice pomiędzy uzyskanymi wynikami.

Uzyskane wyniki wskazują, że wpływ charakterystyki danych wejściowych na czas wykonania był zależny od badanego algorytmu. Porównanie wyników dla danych całkowitych i zmiennoprzecinkowych nie wykazało wyraźnej przewagi żadnego z tych typów dla wszystkich badanych algorytmów. Również obecność duplikatów nie wpływała w taki sam sposób na czas wykonania we wszystkich analizowanych przypadkach.

Największą zmienność wyników wynikającą ze zmiany charakterystyki zbioru zaobserwowano dla Quick Sort, natomiast Bucket Sort i Sample Sort uzyskiwały bardziej zbliżone wyniki dla analizowanych wariantów danych.

Rozdział 7

Wnioski

7.1 Najważniejsze wyniki przeprowadzonych badań

7.2 Interpretacja uzyskanych wyników

7.3 Ograniczenia przeprowadzonych badań

7.4 Kierunki dalszych badań

Podsumowanie

Podsumowanie wyników, najważniejsze wnioski, ocena czy cele pracy zostały osiągnięte, ograniczenia przeprowadzonych badań oraz możliwe kierunki dalszego rozwoju i optymalizacji zaimplementowanych algorytmów.

Bibliografia

- [1] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 3rd Edition, The MIT Press, 2009.
- [2] Božidar D., Dobravec T., *Comparison of parallel sorting algorithms*, Technical report, Faculty of Computer and Information Science, University of Ljubljana, November 2015.
- [3] Maus A., *A full parallel Quicksort algorithm for multicore processors*, Department of Informatics, University of Oslo, 2015.
- [4] Goodrich M.T., Tamassia R., *Algorithm Design and Applications*, Wiley, 2015.
- [5] Cormen T.H., Leiserson C.E., Rivest R.L., Stein C., *Introduction to Algorithms*, 4th Edition, The MIT Press, 2022.
- [6] Hong H., *Parallel Bucket Sorting Algorithm*, Computer Science Department, San Jose State University.
- [7] Bingmann T., Sanders P., *Parallel String Sample Sort*, Algorithms – ESA 2013, Lecture Notes in Computer Science, vol. 8125, Springer, 2013
- [8] Shi H., Schaeffer J., *Parallel Sorting by Regular Sampling*, Journal of Parallel and Distributed Computing, vol. 14, no. 4, 1992
- [9] Grama A., Gupta A., Karypis G., Kumar V., *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003.
- [10] JáJá J., *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992.
- [11] Hennessy J.L., Patterson D.A., *Computer Architecture: A Quantitative Approach*, 4th Edition, Morgan Kaufmann, 2007.
- [12] Intel, *Hyper-Threading Technology*, <https://www.intel.com/content/www/us/en/gaming/resources/hyper-threading.html>, stan na dzień: 08.08.2026.
- [13] Karbowski A., *Amdahl's and Gustafson-Barsis's Laws Revisited*, Pomiary Automatyka Robotyka, 2025.

- [14] Python Software Foundation, *The Python Tutorial*, <https://docs.python.org/3/tutorial/index.html>, stan na dzień: 11.08.2026.
- [15] Python Software Foundation, *multiprocessing – Process-based parallelism*, <https://docs.python.org/3/library/multiprocessing.html>, stan na dzień: 11.08.2026.
- [16] Python Software Foundation, *multiprocessing.shared_memory – Shared memory for direct access across processes*, https://docs.python.org/3/library/multiprocessing.shared_memory.html, stan na dzień: 11.08.2026.
- [17] NumPy Developers, *NumPy documentation*, <https://numpy.org/doc/stable/>, stan na dzień: 11.08.2026.
- [18] The pandas development team, *pandas documentation*, <https://pandas.pydata.org/docs/>, stan na dzień: 11.08.2026.
- [19] Matplotlib Development Team, *Matplotlib documentation*, <https://matplotlib.org/>, stan na dzień: 11.08.2026.
- [20] Rodola G., *psutil documentation*, <https://psutil.readthedocs.io/en/latest/>, stan na dzień: 11.08.2026.
- [21] Python Software Foundation, *The Python Profilers*, <https://docs.python.org/3/library/profile.html>, stan na dzień: 11.08.2026.
- [22] Fredrikson B., *py-spy: Sampling profiler for Python programs*, <https://github.com/benfred/py-spy>, stan na dzień: 11.08.2026.
- [23] JetBrains, *PyCharm – The Python IDE*, <https://www.jetbrains.com/pycharm/>, stan na dzień: 11.08.2026.
- [24] Hipp D.R., *About SQLite*, <https://sqlite.org/about.html>, stan na dzień: 11.08.2026.
- [25] Python Software Foundation, *time – Time access and conversions*, https://docs.python.org/3/library/time.html#time.perf_counter, stan na dzień: 11.08.2026.

Spis rysunków

2.1	Ogólny schemat podziału pracy w sortowaniu równoległym.	8
3.1	Schemat działania Quicksort.	14
3.2	Schemat działania Merge Sort.	15
3.3	Schemat działania Bucket Sort.	17
3.4	Schemat działania Samplesort.	18
6.1	Średni czas wykonania wariantów sekwencyjnych w zależności od rozmiaru zbioru <code>random_int</code>	38
6.2	Średni czas wykonania wariantów równoległych dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.	40
6.3	Średnie wykorzystanie CPU dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.	42
6.4	Średnie wykorzystanie CPU w zależności od rozmiaru zbioru <code>random_int</code> dla 32 jednostek wykonawczych.	44
6.5	Średnie i maksymalne wykorzystanie pamięci RAM dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.	45
6.6	Przyspieszenie badanych algorytmów dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.	48
6.7	Efektywność badanych algorytmów dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów w zależności od liczby jednostek wykonawczych.	49
6.8	Przyspieszenie badanych algorytmów w zależności od rozmiaru zbioru <code>random_int</code> dla 32 jednostek wykonawczych.	51
6.9	Wpływ typu danych i występowania duplikatów na czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych.	54

6.10 Wpływ stopnia częściowego uporządkowania danych całkowitych na średni czas wykonania badanych algorytmów dla 1 000 000 elemen- tów i 8 jednostek wykonawczych.	55
6.11 Wpływ stopnia częściowego uporządkowania danych zmiennoprze- cinkowych na średni czas wykonania badanych algorytmów dla 1 000 000 elementów i 8 jednostek wykonawczych.	56

Spis tabel

6.1	Najlepsze czasy wariantów równoległych w porównaniu z wariantami sekwencyjnymi dla 1 000 000 elementów zbioru <code>random_int</code>	38
6.2	Czas wykonania badanych algorytmów w zależności od liczby jednostek wykonawczych.	39
6.3	Średnie obciążenie CPU oraz wykorzystanie pamięci RAM dla zbioru <code>random_int</code> o rozmiarze 1 000 000 elementów i 8 jednostkach wykonawczych.	42
6.4	Przyspieszenie badanych algorytmów w zależności od liczby jednostek wykonawczych.	47
6.5	Efektywność badanych algorytmów w zależności od liczby jednostek wykonawczych.	49
6.6	Średni czas wykonania badanych algorytmów dla różnych charakterystyk danych wejściowych przy 8 jednostkach wykonawczych.	53

Spis listingów

4.1	Funkcja partycjonowania w Quicksort	22
4.2	Tworzenie procesów w Parallel Quicksort	22
4.3	Funkcja scalania w Merge Sort	23
4.4	Równoległe sortowanie połówek i sekwencyjne scalanie w Parallel Merge Sort	24
4.5	Rozdzielanie elementów do kubeków w Bucket Sort	24
4.6	Przydzielanie grup kubeków procesom w Parallel Bucket Sort	25
4.7	Rozdzielanie elementów do kubeków na podstawie wartości rozdzielających w Samplesort	26
4.8	Równoległe sortowanie grup kubeków w Parallel Samplesort	27
4.9	Generowanie danych częściowo posortowanych	28

Streszczenie

Summary

Słowa kluczowe

algorytmy sortowania;
sortowanie równoległe;
procesory wielordzeniowe;
sortowanie szybkie;
sortowanie przez scalanie;
sortowanie kubełkowe;
Samplesort;
programowanie równoległe;
wydajność;
wykorzystanie CPU;
wykorzystanie RAM;
przyśpieszenie;
efektywność;
skalowalność;
Python;
multiprocessing;
pamięć współdzielona

Keywords

sorting algorithms;
parallel sorting;
multicore processors;
Quicksort;
Merge Sort;
Bucket Sort;
Samplesort;
parallel programming;
performance;
CPU utilization;
RAM usage;
speedup;
efficiency;
scalability;
Python;
multiprocessing;
shared memory

Częstochowa, dd.mm.20rr

Imię i nazwisko: Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Wydział: Informatyki i Sztucznej Inteligencji

Oświadczenie autora pracy dyplomowej*

Oświadczam pod rygorem odpowiedzialności karnej, że złożona przeze mnie praca dyplomowa pt. „*Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach*” jest moim samodzielnym opracowaniem i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Jednocześnie oświadczam, że moja praca (w całości ani we fragmentach) nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w Politechnice Częstochowskiej.

Wyrażam/~~nie wyrażam~~** zgodę/zgody na nieodpłatne wykorzystanie przez Politechnikę Częstochowską całości lub fragmentów wyżej wymienionej pracy w publikacjach Politechniki Częstochowskiej.

.....
podpis studenta

*W przypadku zbiorowej pracy dyplomowej, dołącza się oświadczenia każdego ze współautorów pracy dyplomowej.

*Niepotrzebne skreślić.